



厦门理工学院
XIAMEN UNIVERSITY OF TECHNOLOGY

毕 业 设 计（论文）

中文题目 基于 LangChain 和 LLM 构建的智能
程序设计教学辅导平台

英文题目 **Intelligent Programming Instruction
and Tutoring Platform
Based on LangChain and LLM**

院 系： 软件工程学院

年级专业： 2023 级软件工程（专升本）

姓 名： 何伟杰

学 号： 2312114433

指导教师： 张旭

职 称： 副教授

2025 年 5 月 21 日

毕业设计（论文）诚信声明书

本人郑重声明：在毕业设计（论文）工作中严格遵守学校有关规定，恪守学术规范；我所提交的毕业设计（论文）是本人在张旭指导教师的指导下独立研究、撰写的成果，设计（论文）中所引用他人的文字、研究成果，均已在设计（论文）中加以说明；在本人的毕业设计（论文）中未剽窃、抄袭他人的学术观点、思想和成果，未篡改实验数据。

本设计（论文）和资料若有不实之处，本人愿承担一切相关责任。

学生签名：

2025 年 5 月 21 日

毕业设计（论文）AI 工具使用声明书

在本次写作过程中，为了提高内容生成的效率和质量，在确保符合学术规范的前提下，本人使用了 AI 工具，本人对 AI 工具生成内容负全部责任并确保论文原创性和学术诚信。以下是关于 AI 工具使用的详细说明：

一、工具名称、版本号

1.工具名称：DeepSeek、Claude

2.版本号：DeeSeek-V3、Claude-3.7-Sonnet

二、使用范围及过程

1.文本生成及内容修改：使用 DeepSeek 指导其生成论文的绪论部分内容，输入智能程序设计、AI 辅助教学等关键词，根据生成内容参考并完成了绪论中研究意义和研究内容方法。

2.代码辅助：使用 Claude-3.7-Sonnet 生成前端组件，通过提供页面需求的方式指导其完成部分前端页面的编写。

3.其他：无。

学生签名：

2025 年 5 月 21 日

基于 LangChain 和 LLM 构建的智能程序设计教学辅导平台

【摘要】传统程序设计课程的教学模式通常存在一定的局限性，过于注重理论知识的单向传授与基础编程训练，难以有效培养学生创新能力，由于学生个体的差异，导致个性化教学实施困难，学生在解决复杂问题时往往表现出排错能力的欠缺。随着人工智能技术的快速发展，以大语言模型为代表的新兴技术为程序设计教育带来了创新机遇。借助 LangChain 的框架优势，能够高效地整合多种工具与 LLM 进行交互，通过智能化的交互工具为编程教学提供多个维度的支持。本平台将设计出能够实时分析学生代码并给出改进意见，还能深度解析编程中的概念困惑与逻辑错误，结合知识库实现 RAG 检索帮助大语言模型生成。在传统单体架构中，扩展性方面的不足进一步限制了不同技术的结合，为解决这个技术挑战，本文提出了基于微服务架构的系统设计方案，前端采用 Web 界面实现，通过 HTTP 协议与后端交互；后端系统通过注册中心实现微服务的动态发现与调用。这种架构设计有效提升了系统弹性扩展能力并解决了不同系统的交互问题，确保用户能够获得流畅的体验。

【关键词】 LangChain； RAG； 大模型； 教学辅助； 微服务

Intelligent Programming Instruction and Tutoring Platform Based on LangChain and LLM

【Abstract】 The teaching mode of traditional programming courses usually has certain limitations, focusing too much on one-way transmission of theoretical knowledge and basic programming training, which makes it difficult to effectively cultivate students' innovative ability. Due to individual differences among students, personalized teaching implementation is difficult, and students often show a lack of troubleshooting ability when solving complex problems. With the rapid development of artificial intelligence technology, emerging technologies represented by big language models have brought innovative opportunities for programming education. By leveraging the framework advantages of LangChain, multiple tools can be efficiently integrated to interact with LLM, providing multi-dimensional support for programming teaching through intelligent interactive tools. This platform will be designed to analyze student code in real-time and provide improvement suggestions, as well as deeply analyze conceptual confusion and logical errors in programming. Combined with a knowledge base, it will implement RAG retrieval to help generate large language models. In traditional monolithic architectures, the lack of scalability further limits the integration of different technologies. To address this technological challenge, this paper proposes a system design scheme based on microservice architecture, where the front-end is implemented using a web interface and interacts with the back-end through HTTP protocol; The backend system dynamically discovers and calls microservices through a registry. This architecture design effectively enhances the system's elastic scalability and solves the interaction problems between different systems, ensuring that users can have a smooth experience.

【Keywords】 LangChain; LLM; RAG; Teaching assistance; Microservices

目录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	1
1.2.1 国外研究进展.....	1
1.2.2 国内研究现状.....	2
1.3 研究内容和方法.....	2
1.4 本章小结.....	3
第二章 相关技术分析.....	4
2.1 Spring Cloud.....	4
2.2 FastAPI.....	4
2.3 LLM.....	5
2.4 LangChain.....	5
2.5 RAG.....	5
2.6 本章小结.....	6
第三章 概要设计.....	7
3.1 总体架构设计.....	7
3.1.1 系统物理部署图.....	7
3.1.2 软件应用架构图.....	7
3.2 服务划分.....	8
3.3 数据库设计.....	9
3.4 本章小结.....	17
第四章 详细设计.....	18
4.1 知识库模块.....	18
4.1.1 文档预处理与内容提取.....	18
4.1.2 基于 Embedding 构建知识库.....	23
4.1.3 RAG 流程初步实现.....	27

4.2 聊天应用模块.....	29
4.2.1 聊天模型工厂.....	29
4.2.2 多轮对话记忆聊天应用设计.....	30
4.2.3 基于 Agent 工具调用具体实现.....	31
4.2.4 知识库工具实现全流程 RAG.....	34
4.3 本章小结.....	36
第五章 平台实现.....	37
5.1 知识库实现.....	37
5.1.1 知识库列表页面.....	37
5.1.2 知识库详情页面.....	37
5.2 聊天应用实现.....	39
5.2.1 应用列表页面.....	40
5.2.2 应用详情页面.....	40
5.2.3 聊天流程演示.....	42
5.3 本章小结.....	44
第六章 系统测试.....	45
6.1 接口测试.....	45
6.1.1 前期准备.....	45
6.1.2 测试结果.....	47
6.2 功能测试.....	48
6.3 本章小结.....	53
第七章 总结与展望.....	55
7.1 总结.....	55
7.2 展望.....	55
参考文献.....	56
致 谢.....	57

第一章 绪论

1.1 研究背景及意义

程序设计教学作为计算机学科的基础，传统程序设计课程通常存在以下痛点：①以教师为中心，学生主要是被动接受知识的传授^[1]，难以针对每位学生进行个性化的引导与反馈。②知识更新速度跟不上行业发展变化^[2]。

随着大模型技术的迅速发展，大模型在自然语言中多个领域中展现出强大潜力，本文将利用大模型的语言生成和理解能力，为学生提供个性化的学习支持和指导^[3]，同时对复杂概念和逻辑问题进行智能答疑，大大提升了教师教学效率和学生主动学习的积极性。

平台将从以下技术和架构方面出发：

（1）在技术实现上，平台采用 LangChain 作为 AI 开发框架，它提供了完整的 AI 开发类库，能整合各种 AI 模型与工具，开箱即用，能够便捷的开发大模型应用，同时也提供了实现 RAG 技术的成熟流程，将现有知识库和题库扩展应用于大模型中，这种方式满足复杂问题的解答需求，也提升了教学过程中对最新技术点和案例的实时获取能力。

（2）在系统架构层面，针对传统单体架构较差可扩展性，本平台设计了基于微服务架构的后端系统，微服务之间采用 Restful 轻量级 HTTP 协议相互调用^[4]，进一步保障平台在高负载下的响应性能和横向扩展能力。

总之，本文研究的平台在技术上不仅具有前沿性更在程序设计教学上提供了全新的思路和实践路线，对当前编程教学场景具有借鉴意义。

1.2 国内外研究现状

1.2.1 国外研究进展

在国外，智能教学和编程辅助工具研究方面起步比较早，已经有多种基于人工智能辅助编程教学系统的探索实现。以欧美等地来说，研究者们重点都在关注如何利用深度学习技术实现代码自动生成、代码调试以及在线答疑等功能。例如，由 GitHub 提出的 Copilot 编程工

具，主要面向开发者在代码编辑器（IDEA）中以插件的形式存在，通常以代码补全为核心功能。GitHub Copilot 虽然能够基于自然语言描述生成代码，通过开发者的注释且根据上下文预测代码，但主要面向程序开发者并不适用于教学场景。

1.2.2 国内研究现状

在国内中，近些年大模型教育应用的研究中，政策支持与技术落地并重，科技部等六部门印发的《关于加快场景创新以人工智能高水平应用促进经济高质量发展的指导意见》明确提出着力打造人工智能重大场景，鼓励高校院所参与场景创新。

在企业中，阿里云推出的通义灵码与 Copilot 类似虽支持代码补全，但同样也缺乏教学上需要的交互设计。在高校实践中，全国高等教育领域首个编程大模型教学应用“码上”在北邮教学云平台上线。“码上”由北京邮电大学国际学院 EZ Coding 大学生创业团队打造，该平台为学生提供 1 对 1 辅导服务，为教师提供多维度的教学支持服务，目的是达到提高学生学习效率减轻教师工作负担。

总体来看，无论国内外大多数程序设计辅助仍停留在工具层面，缺乏对程序设计教学场景的深度定制，针对程序设计教学中学和练结合等需求的功能集成不足。因此围绕程序设计教学场景，结合 LLM 与 LangChain 开发具有自主交互能力与可扩展性的教学辅导平台，具有广阔的研究空间。

1.3 研究内容和方法

本论文将主要围绕基于 LangChain 和 LLM 构建智能程序设计教学辅导平台展开研究，主要内容和方法如下：

架构设计与技术选型：

选择 LangChain 作为 AI 开发核心框架，结合 LLM 实现自然语言处理和生成，同时设计基于微服务架构的后端系统，确保平台具有良好的扩展能力。

关键技术实现：

- （1）如何设计前后端通信接口，以及如何划分各服务；
- （2）运用 RAG 技术构建题库扩充与知识检索模块，支持学生自主学习；
- （3）利用 LangChain 的提示模板与聊天消息列表，实现多轮对话记忆，并记录学生聊天

历史，以提供连续性指导。

在方法论上，本研究采用文献调研、系统设计、平台实现、平台测试相结合的方式，尝试从理论和实践两方面论证平台实现的可行性与应用价值。

1.4 本章小结

本章分析了传统程序设计教学的局限性，阐明了 LLM 与 LangChain 技术融合的教育和技术价值，通过国内外研究现状，指出当前 LLM 教育应用的新方向，进一步提出以 LangChain 框架为核心、微服务架构为支撑的系统设计方案，为后续章节的技术实现与实验验证奠定理论基础。后续将围绕本平台涉及的技术栈展开详细探讨。

第二章 相关技术分析

2.1 Spring Cloud

Spring Cloud Alibaba 是基于 Spring Cloud 生态系统开发的分布式系统解决方案，结合 Spring Cloud 自身生态和阿里自研组件，提供完整的分布式系统开发框架。其核心优势在于对构建高可用性和灵活扩展性项目的支持，从服务注册与发现、配置中心、消息总线到分布式限流与链路追踪，Spring Cloud Alibaba 实现了分布式场景的覆盖，可以帮助开发者快速搭建出微服务平台。

Spring Cloud 的各个模块之间关联不大，开发人员可以根据项目需求来灵活挑选技术选型^[5]。在本项目中，借助 Spring Cloud 和 Spring Cloud Alibaba 通过以下组件实现核心功能：

(1) Gateway 作为所有微服务的网关，统一整个系统的请求入口，主要功能就是进行路由转发和跨域处理，不同的服务部署在不同的集群和地区，Gateway 网关也可配合服务注册与发现组件，实现负载均衡的功能。

(2) Nacos 作为服务注册与配置中心，支持多实例负载均衡和服务健康检查。例如，前端调用通用的网关请求，网关将请求转发至对应服务实例处理，其中对应服务的 IP 以及端口号使用 Nacos 获取，解决多服务通信问题。

(3) OpenFeign 通过注解驱动的方式简化服务间 HTTP 调用。同时集成 Ribbon 实现客户端负载均衡。

2.2 FastAPI

FastAPI 是基于 Python 的现代 Web 框架，利用 Starlette 作为底层异步框架以异步非阻塞 IO 和高性能著称。FastAPI 特性包括：基于开放标准、自动生成文档、更主流的 Python、依赖注入等^[6]。

在本平台中，其核心作用包括：

(1) 高效通信：作为微服务集群的一员，FastAPI 通过异步请求处理实现用户与大模型进行实时对话交互。

(2) 接口标准化：采用 OpenAPI 规范定义 RESTful 接口，便于与 Spring Cloud 微服务后

端协同开发。例如，接口通过 Pydantic 模型验证输入数据，减少接口调用失败率。

相较于传统框架，FastAPI 的自动文档生成与类型提示符合现代编程理念，可以显著提升开发效率，适合快速迭代的人工智能应用。

2.3 LLM

大型语言模型（LLM）包括如 GPT-4、DeepSeek、通义千问等，凭借它们强大能力，已在文本生成、语言理解、对话聊天等诸多领域中展示出显著优势。本项目的 LLM 应用聚焦以下方向：

（1）代码逻辑解析：能够解答用户在编码过程中遇到的概念性和逻辑性问题，还可以通过上下文理解给出代码建议和优化措施。

（2）交互式问题解答：利用 LLM 的多轮对话能力，模拟教师辅导过程，引导用户逐步解决编程问题。

2.4 LangChain

随着多种大模型发展，Lang Chain 框架应运而生，框架生来就是为了支持 LLM 应用开发^[7]，核心理念是：简化大语言模型应用开发的流程。本平台中关键技术点包括：

（1）工具链（Chains）：定义任务处理流程为链后续直接调用即可，例如将代码解析、知识库检索与工具调用，串联为完整链路。

（2）智能体（Agents）：支持自主调用工具或数据库，例如在解答问题时查询官方文档或知识库中的内容。

（3）状态（State）：记忆用户历史聊天记录，实现多轮会话的连续性指导，例如跟踪上下文寻找关联的内容以供回答。

2.5 RAG

检索增强生成（Retrieval Augmented Generation）是一种随着大模型而流行的检索技术，其理念是将信息检索技术与大模型相结合。其核心思想是：在生成回答前先进行相关信息检索，再将检索结果作为模型上下文输入，以提供模型获得更准确、更具针对性的答案。RAG 技术能够有效解决 LLM 在特定领域知识不足的问题^[8]，也就是避免 LLM 的“幻觉”问题，

具体表现在缓解大语言模型在处理最新知识时的知识滞后问题，特别适用于知识更新频繁或领域专业性较强的应用场景。

对于程序设计教学而言，RAG 技术可以在大语言模型的基础上构建专业领域内的知识库，进一步提升问答的准确率。具体来说，当学生提出问题时，系统会先通过检索模块找到与问题相关的文档块或者代码实例，再借助 LLM 生成归纳总结，实现精准答复的效果，避免模型直接编造不存在的内容。

2.6 本章小结

本章从相关技术出发，分析了对本项目中关键技术——Spring Cloud、FastAPI、大语言模型、LangChain 以及 RAG。其中，分布式架构和高性能接口作为基础设施构建保证了系统的稳定与扩展；而基于大语言模型与 LangChain 为核心框架，则为提升教学效率和学生体验奠定了技术基础。后续章节将基于此技术框架，阐述平台详细设计与系统实现过程。

第三章 概要设计

3.1 总体架构设计

3.1.1 系统物理部署图

本平台采用前后端分离开发模式，前端由 Vue3 驱动，后端为微服务集群。前端采用 Nginx 作为代理访问后端微服务网关。后端主要为一系列划分了职责的服务实例组成，也包含了基础设施服务（MySQL、Redis、Elasticsearch），其中 MySQL 负责数据的持久化、Redis 负责缓存数据、Elasticsearch 作为向量数据库、服务由一些系列 web 实例构成。详细系统物理部署图如图 3.1 所示：

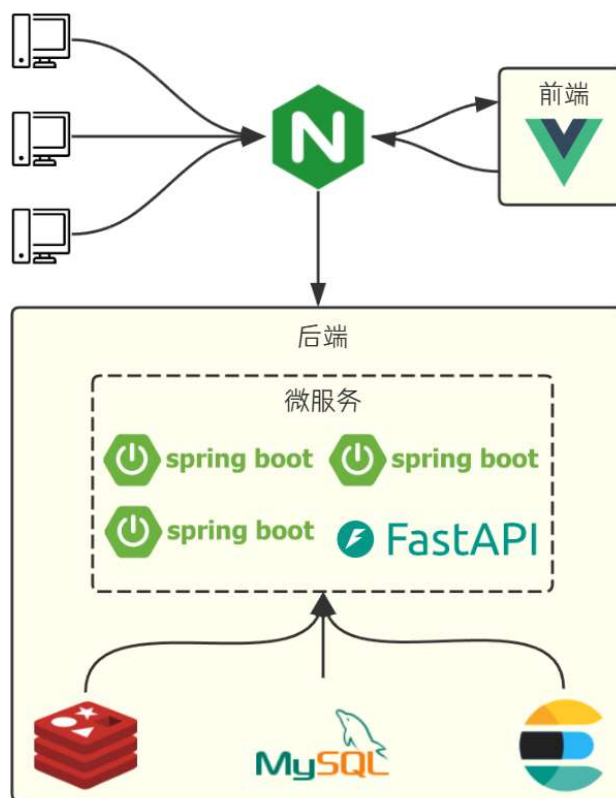


图 3.1 系统物理部署图

3.1.2 软件应用架构图

本平台采用 MVC 三层架构设计理念，软件应用架构图如图 3.2 所示：

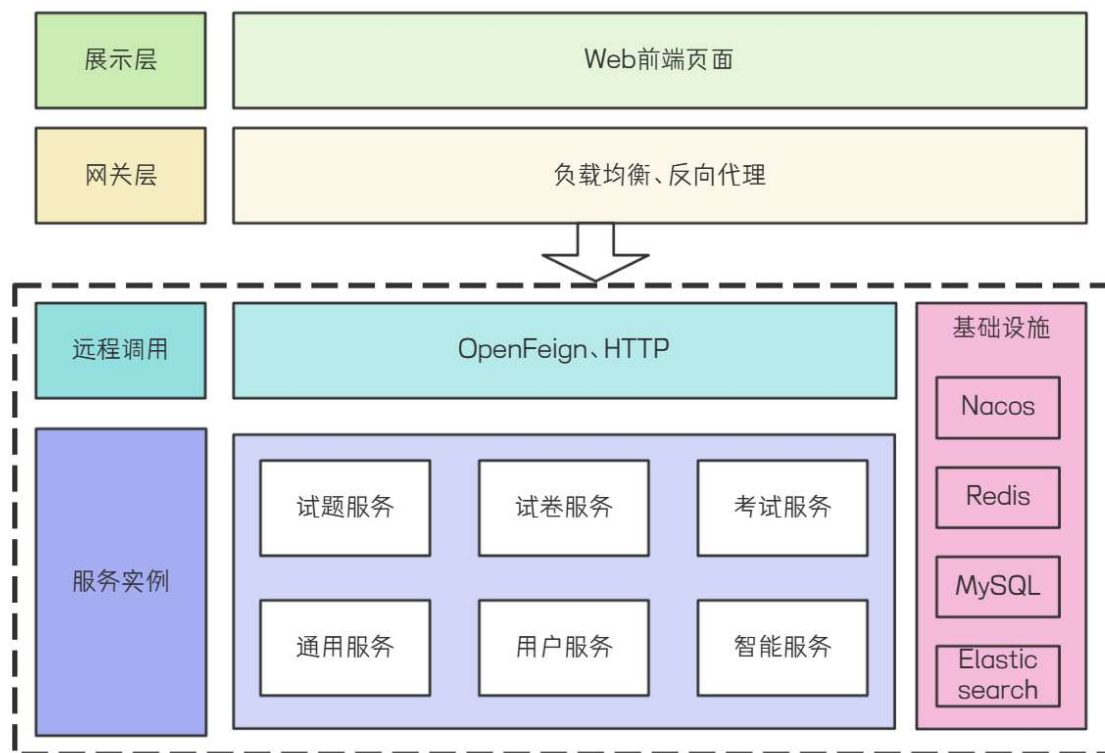


图 3.2 软件应用架构图

- (1) 展示层：提供与用户交互的 Web 前端页面。
- (2) 网关层：分发请求，提供负载均衡、反向代理功能。
- (3) 远程调用层：为服务实例之间相互通信做桥梁。各服务通过 HTTP 协议进行数据传输，以达到远程调用的目的。
- (4) 服务实例层：通过职责划分出最小粒度的服务实例，处理对应的业务逻辑。
- (5) 基础设施层：由多个中间件构成，为整个系统提供基础设施服务。如：数据存储、高速缓存、服务发现、配置管理等。

3.2 服务划分

本项目采用了微服务技术，通过合理的服务拆分平衡系统的复杂性，故采用单一职责原则（SRP），即每个服务只负责明确的业务功能，当功能或业务变更时仅影响单个服务。同时在进行划分时需避免过度拆分，注意服务粒度的平衡，服务不宜过大（失去微服务优势）或过小（分布式事务问题）。

基于上述的原则，本项目主要划分了 6 个微服务，分别为：通用服务、用户服务、智能

服务、试题服务、试卷服务、考试服务。

(1) 通用服务: 主要向其他服务提供通用业务的外部接口, 如向试题服务提供学科信息、向用户服务提供院校认证信息、提供文件上传接口等。

(2) 用户服务: 主要负责提供平台的所有用户相关业务, 如用户认证与授权、用户组管理等。

(3) 智能服务: 核心服务, 主要负责知识库、聊天应用等相关业务。

(4) 试题服务: 主要负责平台题库的管理。

(5) 试卷服务: 主要处理试卷业务, 包括手动组卷、智能组卷、试卷归档等。

(6) 考试服务: 主要负责考试全流程管理, 例如考试安排、成绩管理等。

通过合理的服务划分方式, 遵循了单一职责原则, 每个服务聚焦特定业务领域, 实现松耦合。各微服务之间通过 Nacos 实现服务注册与动态发现, 外部请求统一 Gateway 处理外部请求并路由到对应服务, 通过 OpenFeign 实现服务之间 Restful 远程调用, 最终实现协同工作。

3.3 数据库设计

确定整个系统架构和划分具体服务之后, 可以确定基本的存储对象单元, 由此抽象出基本数据库表结构。本项目涉及的数据表概览如表 3.1 所示:

表 3.1 数据表概览

序号	表名	说明
1	user	用户基础表
2	user_info	用户信息表
4	user_group	用户组表
5	user_group_users	用户组关联用户表
6	subject	学科表
7	question	试题主体表
8	question_option	试题选项表
9	knowledge_base	知识库表
10	knowledge_file	知识库文档表

续表 3.1

序号	表名	说明
11	file_document	源文件-文档块关联表
12	application	应用表
13	application_settings	应用配置表
14	conversation	会话表
15	history_messages	历史消息记录表

完整的数据库表结构如表 3.2 至表 3.15 所示。

表 3.2 用户基础表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	用户 id
phone	char	11	N	手机号
nickname	varchar	20	N	昵称
avatar	varchar	256	N	头像
role	tinyint	4	N	角色类型
password	varchar	256	N	密码
status	tinyint	4	N	用户状态
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	4	N	是否删除

表 3.3 用户信息表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	用户 id
real_name	varchar	10	N	真实姓名
sex	tinyint	3	N	性别
description	varchar	50	N	简介

续表 3.3

字段名	数据类型	长度	允许空值	注释
email	varchar	256	N	邮箱
id_card	varchar	256	N	身份证号
birth	date	10	N	生日
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	4	N	是否删除

表 3.4 用户组表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	用户组 id
did	bigint	20	N	院校 id
type	tinyint	4	N	用户组类型
title	varchar	20	N	用户组名称
description	varchar	50	N	用户组描述
creator	bigint	20	N	创建者 id
create_time	datetime	19	N	创建时间
update_time	datetime	19	Y	更新时间
is_deleted	tinyint	4	N	是否删除

表 3.5 用户组关联用户表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	主键 id
gid	bigint	20	N	关联的用户组 id
uid	bigint	20	N	关联的用户 id
create_time	datetime	19	N	创建时间

续表 3.5

字段名	数据类型	长度	允许空值	注释
update_time	datetime	19	N	更新时间
is_deleted	tinyint	4	N	是否删除

表 3.6 学科表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	学科 id
name	varchar	20	N	学科名称
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.7 试题主体表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	试题 id
subject_id	bigint	20	N	学科 id
type	tinyint	3	N	试题类型
level	tinyint	3	N	难度
content	varchar	1024	N	试题题目
creator	bigint	20	N	创建人
analysis	varchar	50	Y	解析
is_public	tinyint	3	N	是否公开
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	时间
is_deleted	tinyint	3	N	是否删除

表 3.8 试题选项表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	选项 id
question_id	bigint	20	N	试题 id
is_right	tinyint	3	N	是否为正确选项
content	varchar	128	N	选项内容
sort	tinyint	3	N	排序
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.9 知识库表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	知识库 id
kb_name	varchar	50	N	知识库名称
kb_info	varchar	200	N	知识库介绍
vs_type	varchar	50	N	向量数据库类型
user_id	bigint	20	N	所属用户 id
embed_model	varchar	50	N	嵌入模型
k	int	10	N	检索返回的最终数量
fetch_k	int	10	N	用于检索的候选数量
score_threshold	float	13	N	最低相关数阈值
file_count	int	10	N	源文件数量
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.10 知识库文档表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	文档 id
kb_id	bigint	20	N	知识库 id
user_id	bigint	20	N	文档归属用户 id
file_name	varchar	50	N	文档名称
file_ext	varchar	20	N	文件后缀名
file_size	bigint	20	N	文档大小
version	int	10	N	版本号
document_loader	varchar	128	N	文档加载器
document_count	int	10	N	文档计数
text_splitter	varchar	128	N	文档分割器
in_folder	bit	1	N	是否有源文件
in_vector_db	bit	1	N	在向量数据库中
index_name	varchar	128	N	索引名称
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	4	N	是否删除

表 3.11 源文件-文档块关联表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	主键 id
kb_id	bigint	20	N	知识库 id
file_id	bigint	20	N	文件 id
docs_id	varchar	128	N	分割后的文档块 id
meta_data	varchar	1024	N	元数据
create_time	datetime	19	N	创建时间

续表 3.11

字段名	数据类型	长度	允许空值	注释
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.12 应用表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	appid
kb_id	bigint	20	N	关联的知识库 id
creator_id	bigint	20	N	创建者 id
name	varchar	20	N	app 名称
cover	varchar	1024	N	应用封面
introduction	varchar	100	N	应用介绍
status	tinyint	3	N	状态
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.13 应用配置表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	主键 id
app_id	bigint	20	N	关联的 appid
chat_model	varchar	50	N	聊天模型
prompt	varchar	1024	N	提示词
prologue	varchar	1024	N	开场白
temperature	float	12	N	采样温度
max_tokens	int	10	N	回复最大 token 数
stream	bit	1	N	流式响应

续表 3.13

字段名	数据类型	长度	允许空值	注释
k	int	10	N	检索返回的最终数量
fetch_k	int	10	N	用于检索的候选数量
score_threshold	float	12	N	最低相关数阈值
knowledge_bases	varchar	1024	N	关联的知识库
tools	varchar	1024	N	关联的工具
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.14 会话表

字段名	数据类型	长度	允许空值	注释
id	varchar	512	N	会话 id
creator_id	bigint	20	N	创建者 id
app_id	bigint	20	N	产生会话的 appid
title	varchar	128	N	会话标题
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

表 3.15 历史消息记录表

字段名	数据类型	长度	允许空值	注释
id	bigint	20	N	消息 id
conversation_id	varchar	512	N	会话 id
query	text	65535	Y	用户提问
response	text	65535	Y	响应内容
response_metadata	json	65535	Y	响应元数据

续表 3.15

字段名	数据类型	长度	允许空值	注释
usage_metadata	json	65535	Y	消息用量元数据
create_time	datetime	19	N	创建时间
update_time	datetime	19	N	更新时间
is_deleted	tinyint	3	N	是否删除

3.4 本章小结

本章主要描述了本项目的总体架构设计和物理设计。一是确定了各层之间的依赖调用关系，并且使用绘图的形式从部署和模块的角度出发进行说明。二是对系统设计的模块进行服务划分并简述分析了其划分依据。三是按整体架构图建立数据库表结构，通过表格的形式展现。

第四章 详细设计

4.1 知识库模块

本小节的职责是负责设计和构建知识库。程序设计文档的特点通常是已非结构化的形式存在，包括不同格式的文档、书籍等，这涉及到如何将知识点从非结构化的海洋中提取出来构建知识库的问题。

本平台的知识库构建包括：文档预处理、内容提取、知识存储三步骤。

(1) 文档预处理是构建高质量检索的关键，从源文档的类型角度出发涉及多种格式，如：Word 文档 (.docx)、PDF 文件 (.pdf)、Markdown (.md) 以及图片 (.jpg) 等格式。对于如何处理不同类型的文档将在下文进行详细讨论并实现。

(2) 内容提取包括两部分：①文档内容分块（拆分）、②元数据提取。一、文档内容分块指的是从一个具体的文档文件中将其内容也就是文本进行分段处理，文档内容也存在多种格式例如文本、表格、插图等。首先需要确定的是在提取时，不关心文档内容的格式，只需要关注其内容本身即可。文档内容的拆分是一个值得讨论的点，其拆分过程涉及文档块的大小、上下文语义的连贯性等问题。二、元数据提取指的是从文档的本身提取标题、作者、日期等属于文档自身的属性，以便于后续的知识检索和组织。

(3) 知识存储是构建知识库的关键，其涉及如何存储拆分后的文档块以及文档块存储组织方式。

总的来说，知识库从 0 到 1 的构建过程包含如上所述的三部曲，下文将对于每一步骤涉及的问题进行讨论并实现。

4.1.1 文档预处理与内容提取

文档预处理简单来说就是把原始文档转换为结构化的、易于检索和理解的格式。本文讨论的文档格式包括非结构化或半结构化（Markdown、PDF、Word、PPT、Image）。首先需要针对不同的格式设计处理不同的文档加载器。为了兼容多种文档格式，使用 LangChain 包下提供的 UnstructuredFileLoader 非结构化文件加载器重写实现了 PDF 文档加载器、Word 文档加载器、PPT 文档加载器、Image 图片加载器。Word 文档加载器具体实现如图 4.1 所示：

```
class RapidOCRDocLoader(UnstructuredFileLoader):
    @shuofeng *
    def _get_elements(self) -> list:
        @shuofeng *
        def doc2text(filepath):

            ocr = RapidOCR()
            doc = Document(filepath)
            resp = ""

            @shuofeng
            def iter_block_items(parent):
                from docx.document import Document

                if isinstance(parent, Document):
                    parent_elm = parent.element.body
                elif isinstance(parent, _Cell):
                    parent_elm = parent._tc
                else:
                    raise ValueError("RapidOCRDocLoader parse fail")

                for child in parent_elm.iterchildren():
                    if isinstance(child, CT_P):
                        yield Paragraph(child, parent)
                    elif isinstance(child, CT_Tbl):
                        yield Table(child, parent)

            b_unit = tqdm.tqdm(
                total=len(doc.paragraphs) + len(doc.tables),
                desc="RapidOCRDocLoader block index: 0",
            )
            for i, block in enumerate(iter_block_items(doc)):
                b_unit.set_description("RapidOCRDocLoader block index: {}".format(i))
                b_unit.refresh()
                if isinstance(block, Paragraph):
                    resp += block.text.strip() + "\n"
                    images = block._element.xpath("./pic:pic") # 获取所有图片
                    for image in images:
                        for img_id in image.xpath("./a:blip/@r:embed"): # 获取图片id
                            part = doc.part.related_parts[
                                img_id
                            ] # 根据图片id获取对应的图片
                            if isinstance(part, ImagePart):
                                image = Image.open(BytesIO(part._blob))
                                result, _ = ocr(np.array(image))
                                if result:
                                    ocr_result = [line[1] for line in result]
                                    resp += "\n".join(ocr_result)
                elif isinstance(block, Table):
                    for row in block.rows:
                        for cell in row.cells:
                            for paragraph in cell.paragraphs:
                                resp += paragraph.text.strip() + "\n"
                b_unit.update(1)
            return resp

        text = doc2text(self.file_path)
        from unstructured.partition.text import partition_text

        return partition_text(text=text, **self.unstructured_kwargs)
```

图 4.1 Word 文档加载器具体实现代码

对于其他类型的文档都使用 LangChain 自带的加载器 TextLoader 进行文档加载，特别的 CSV 文件使用 CSVLoader 来加载。最后设计统一获取文档加载器函数，支持传入加载器名称和文档路径得到对应类型的加载器实例。

具体的代码实现如下图 4.2 所示：

```
def get_loader(loader_name: str, file_path: str, loader_kwargs: dict = None) -> BaseLoader:
    """
    根据loader_name和文件路径返回文档加载器
    """
    loader_kwargs = loader_kwargs or {}
    try:
        if loader_name in [
            "RapidOCRPDFLoader",
            "RapidOCRLoader",
            "RapidOCRDocLoader",
            "RapidOCRPTLoader",
        ]:
            document_loaders_module = importlib.import_module(
                "app.service.document_loader"
            )
        else:
            document_loaders_module = importlib.import_module(
                "langchain_community.document_loaders"
            )
        DocumentLoader = getattr(document_loaders_module, loader_name)
    except Exception as e:
        msg = f"为文件{file_path}查找加载器{loader_name}时出错"
        log.error(f'{msg}: {e}')
        raise BusinessException(msg)

    if loader_name == "CSVLoader" or loader_name == "TextLoader":
        if not loader_kwargs.get("encoding"):
            # 如果未指定 encoding，自动识别文件编码类型，避免langchain loader 加载文件报编码错误
            with open(file_path, "rb") as struct_file:
                import chardet
                encode_detect = chardet.detect(struct_file.read())
            if encode_detect is None:
                encode_detect = {"encoding": "utf-8"}
            loader_kwargs["encoding"] = encode_detect["encoding"]

    return DocumentLoader(file_path, **loader_kwargs)
```

图 4.2 获取加载器实现代码

由于语言模型的输入窗口是有限的，没办法将一整篇本地的文档直接输入给语言模型，所以说需要先将文档切割为较短的片段，然后从中筛选出与提问有关的文档片段，最后再结合具体的提问内容一起提供给语言模型^[6]。使用文档加载器提取文档内容后，得到内容文本还需进一步的拆分处理，LangChain 框架默认提供 RecursiveCharacterTextSplitter 递归字符文本

分割器进行处理。该分割器涉及两个关键参数：`chunk_size`、`chunk_overlap`。第一个 `chunk_size` 参数表示拆分之后每块的大小，第二个 `chunk_overlap` 参数表示文档块之间的重叠数。在实际的使用过程中，还需要实时调整两个参数，控制拆分后文档块质量。

由于 LangChain 框架中提供的 `RecursiveCharacterTextSplitter` 对中文的拆分支持不够完善，本平台自定义实现了对中文标点符号更加优化的分割器 `ChineseRecursiveTextSplitter`。具体的实现代码如图 4.3 至图 4.4 所示：

```
class ChineseRecursiveTextSplitter(RecursiveCharacterTextSplitter):
    """shuofeng"""
    def __init__(
        self,
        separators: Optional[List[str]] = None,
        keep_separator: bool = True,
        is_separator_regex: bool = True,
        **kwargs: Any,
    ) -> None:
        super().__init__(keep_separator=keep_separator, **kwargs)
        self._separators = separators or [
            "\n\n",
            "\n",
            "。 ! ? ",
            "\. \s|!| \s|? \s",
            ": |; \s",
            ", |, \s",
        ]
        self._is_separator_regex = is_separator_regex

    1 usage """shuofeng"""
    def _split_text(self, text: str, separators: List[str]) -> List[str]:
        """Split incoming text and return chunks."""
        final_chunks = []
        # 获取要使用的适当分隔符
        separator = separators[-1]
        new_separators = []
        for i, _s in enumerate(separators):
            _separator = _s if self._is_separator_regex else re.escape(_s)
            if _s == "":
                separator = _s
                break
            if re.search(_separator, text):
                separator = _s
                new_separators = separators[i + 1:]
                break
```

图 4.3 中文递归解析器具体实现代码 1

```

_separator = separator if self._is_separator_regex else re.escape(separator)
splits = _split_text_with_regex_from_end(text, _separator, self._keep_separator)
# 现在开始合并内容，递归地拆分较长的文本。
_good_splits = []
_separator = "" if self._keep_separator else separator
for s in splits:
    if self._length_function(s) < self._chunk_size:
        _good_splits.append(s)
    else:
        if _good_splits:
            merged_text = self._merge_splits(_good_splits, _separator)
            final_chunks.extend(merged_text)
            _good_splits = []
        if not new_separators:
            final_chunks.append(s)
        else:
            other_info = self._split_text(s, new_separators)
            final_chunks.extend(other_info)
if _good_splits:
    merged_text = self._merge_splits(_good_splits, _separator)
    final_chunks.extend(merged_text)
return [
    re.sub(pattern: r"\n{2,}", repl: "\n", chunk.strip())
    for chunk in final_chunks
    if chunk.strip() != ""
]

```

图 4.4 中文递归解析器具体实现代码 2

同样地，设计一个函数根据传入的分割器名称和分割参数（`chunk_size` 和 `chunk_overlap`）得到对应的文档分割器。对 Markdown 文档拆分进行特殊处理，使用标题头进行拆分，因为 Markdown 是轻量级标记语言默认使用不同级别标题头来组织和区分内容。具体代码实现如下图 4.5 至图 4.6 所示：

```

def get_text_splitter(splitter_name: str, chunk_size: int = 4000, chunk_overlap: int = 200) -> TS:
    try:
        # MarkdownHeaderTextSplitter特殊判定
        if splitter_name == "MarkdownHeaderTextSplitter":
            text_splitter = MarkdownHeaderTextSplitter(
                # 根据不同的标题级别拆分
                headers_to_split_on=[("#", "header1"), ("##", "header2"), ("###", "header3"),
                                     ("####", "header4"), ("#####", "header5"), ("#####", "header6")],
                return_each_line=False,
                # 拆分的内容是否剥离标题
                strip_headers=False
            )
    
```

图 4.5 获取文档分割器具体实现代码 1

```

else:
    try:
        # 优先使用用户自定义的text_splitter
        text_splitter_module = importlib.import_module("app.service.text_splitter")
        TextSplitter = getattr(text_splitter_module, splitter_name)
    except:
        # 否则使用langchain的text_splitter
        text_splitter_module = importlib.import_module(
            "langchain_text_splitters"
        )
        TextSplitter = getattr(text_splitter_module, splitter_name)
    # 从tiktoken加载
    text_splitter = TextSplitter.from_tiktoken_encoder(
        encoding_name="gpt-4",
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
    )
except:
    # 报错则默认使用RecursiveCharacterTextSplitter
    text_splitter_module = importlib.import_module("langchain_text_splitters")
    TextSplitter = getattr(text_splitter_module, "RecursiveCharacterTextSplitter")
    text_splitter = TextSplitter(chunk_size=chunk_size, chunk_overlap=chunk_overlap)
return text_splitter
    
```

图 4.6 获取文档分割器具体实现代码 2

4.1.2 基于 Embedding 构建知识库

为了解决知识存储问题，本平台知识库将基于 Embedding 技术将文档块向量化并存储到向量数据库的方式，下文将开始介绍 Embedding 技术和向量数据库存储设施。

Embedding 是指将高维度数据映射到低维度空间的过程，用于衡量文本字符串的相关性。简单来说，在对文档预处理之后需对分块的文档块贴上特殊的标签，这个过程就叫做嵌入向量。生成这个向量的工具称为嵌入模型，嵌入模型可以把文档块转换为高维度向量。之后再进行搜索时，通过直接比对向量得到原始文档块。这种方式大大减少了模型的计算量，能在有限的成本中找到关心的重点内容。具体流程如图 4.7 所示：

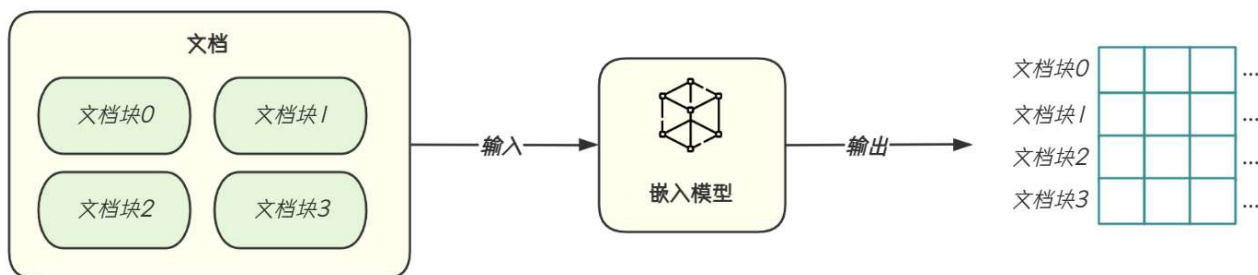


图 4.7 Embedding 技术流程

通过嵌入模型得到的高维向量需要存储到专门的向量数据库中，本平台使用 FAISS 和 Elasticsearch 两款向量数据库作为后续 RAG 检索的存储设施。所谓向量数据库就是能存储和支持检索高维向量的数据库系统，向量数据库不仅存储了高维度向量还保留了原始的文档块和元数据信息，方便在检索召回时组装结果。

考虑到不同知识库可能使用不同的向量数据库作为存储设施，在这里使用抽象类定义出公共知识库服务类，将基本的知识库行为，例如：创建、删除和修改知识库信息等行为规范到抽象类中。将检索、添加、删除和查询文档块等具体实现细节交给子类自行实现，这样做的好处是方便扩展、支持快速接入等，在软件设计中达到松耦合的效果。根本原因是不同的向量数据库在检索和嵌入文本块实现的具体逻辑是不相同的。

对于本平台而言，支持 FAISS 和 Elasticsearch 两款向量数据库两者的检索和存储逻辑是存在差异的。针对两款不同向量数据库，分别实现上述步骤中定义的公共知识库服务类，具体的类继承关系如图 4.8 所示：



图 4.8 不同类型知识库继承关系图

后续通过工厂设计模式的编码，通过传入 Id 从知识库工厂类得到对应的知识库实例。具体实现如图 4.9 所示：

```
class KBServiceFactory:
    10 usages  shuofeng *
    @staticmethod
    def get_service_by_id(kb_id: int) -> KBService | None:
        """
        通过kb_id获取知识库实例
        """
        # 从数据库中加载知识库信息
        kb_id, kb_name, kb_info, user_id, embed_model, vs_type = load_kb_from_db(kb_id)
        if kb_name:
            return KBServiceFactory.get_service(kb_id, kb_name, kb_info, user_id, embed_model, vs_type)
        else:
            return None

    2 usages  shuofeng *
    @staticmethod
    def get_service(kb_id: int, kb_name: str, kb_info: str, user_id: int, embed_model: str, vs_type: str) -> KBService:
        try:
            # 通过反射提取支持的vs类型
            vs_type = getattr(SupportedVSType, vs_type.upper())
        except:
            vs_type = "unknown"
        params = {
            "kb_id": kb_id,
            "kb_name": kb_name,
            "kb_info": kb_info,
            "user_id": user_id,
            "embed_model_name": embed_model
        }
        # 如果向量数据库类型是ES, 则返回ES类型的知识库
        if vs_type == SupportedVSType.ELASTICSEARCH:
            return ElasticsearchKBService(**params)
        # 如果向量数据库类型是FAISS, 则返回FAISS类型的知识库
        elif vs_type == SupportedVSType.FAISS:
            return FaissKBService(**params)
        else:
            info = f"不支持的向量数据库类型: {vs_type}"
            log.error(info)
            raise Exception(info)
```

图 4.9 知识库工厂类

完成知识库的构建之后，为了支持知识库添加文档块的需求，本平台将利用 Embedding 技术向量化文档块并存储至知识库对应的向量数据库中。

以下是本平台知识库添加文档块流程，具体实现代码如图 4.10 所示：

- (1) 选择知识库，上传文档文件。
- (2) 根据文档加载器和分割器，加载并分割得到文档块。
- (3) 为分割后的文档块添加元数据信息。
- (4) 根据知识库对应的向量数据库调用底层向量化方法，并返回文档块 id。
- (5) 建立文档块 id 与向量映射关系，并持久化。
- (6) 更新文档文件状态，完成文档块添加操作。

```
@router.post(path: '/add_docs/{id}', summary='添加文档块接口')
def add_docs(
    file_id: int = Path( default: ..., alias="id", description="文件id"),
    kb_id: int = Body( default: ..., alias="kbId", description="知识库id"),
    chunk_size: int = Body( default: ..., alias="chunkSize", description="分块大小"),
    chunk_overlap: int = Body( default: ..., alias="chunkOverlap", description="分块重叠度"),
    document_loader_name: str = Body( default: ..., alias="documentLoader", description="文档加载器"),
    text_splitter_name: str = Body( default: ..., alias="textSplitter", description="文本分割器"),
):
    kb = KBServiceFactory.get_service_by_id(kb_id)
    if not kb:
        return APIResponse.fail(message="知识库不存在")
    kb_file = get_file_by_id(file_id=file_id)
    if not kb_file:
        return APIResponse(message="源文件不存在")
    # 构造文件路径
    file_path = get_kb_file_path(
        kb_name=kb.kb_name,
        kb_file_name=kb_file.file_name,
        user_id=kb_file.user_id
    )
    # 获得文档加载器
    document_loader = get_loader(
        loader_name=document_loader_name,
        file_path=file_path
    )
    # 获得文档分割器
    text_splitter = get_text_splitter(
        splitter_name=text_splitter_name,
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap
    )
    # 针对markdown的特殊处理
    if text_splitter_name == "MarkdownHeaderTextSplitter":
        md_text = ''.join([doc.page_content for doc in document_loader.load()])
        docs: List[Document] = text_splitter.split_text(md_text)
        for index, doc in enumerate(docs):
            # 获取文档长度
            doc_len = len(doc.page_content)
            if doc_len > 2000:
                for i in range((doc_len // 2000) + 1):
                    # 截取段, 并插入到列表
                    docs.insert(index + i + 1, Document(
                        page_content=doc.page_content[i * 2000:i * 2000 + 2000],
                        metadata=doc.metadata
                    ))
            else:
                docs.pop(index)
    else:
        # 加载文档并拆得到文档块
        docs = document_loader.load_and_split(
            text_splitter=text_splitter
        )
    # 为文档添加元数据
    for doc in docs:
        doc.metadata["source"] = file_path
        doc.metadata["source_file_id"] = kb_file.id
        doc.metadata["source_file_name"] = kb_file.file_name
    # 向量化文档块 并添加到知识库
    doc_ids = kb.add_kb_docs(
        kb_file_id=kb_file.id,
        docs=docs
    )
    # 更新知识库文件对象
    update_file_to_db(
        file_id=kb_file.id,
        document_loader=document_loader_name,
        document_count=len(doc_ids),
        text_splitter=text_splitter_name,
        in_vector_db=(1 if len(doc_ids) else 0)
    )
    return APIResponse.ok(message="加载完成", data=doc_ids)
```

图 4.10 知识库添加文档块接口

4.1.3 RAG 流程初步实现

检索增强生成是一个具体的流程，通过语义匹配从向量数据库中检索相关信息，然后将其馈送到 LLMs 进行响应生成^[10]。特别是在专业领域强、知识源更新频繁和时效性强的场景，RAG 能帮助大模型更好的回答用户。经典 RAG 基础流程包括：①向量化用户问题、②相似性检索、③根据检索结果构建上下文、④大模型总结输出。

本平台提出了如下图 4.11 所示的 RAG 流程：

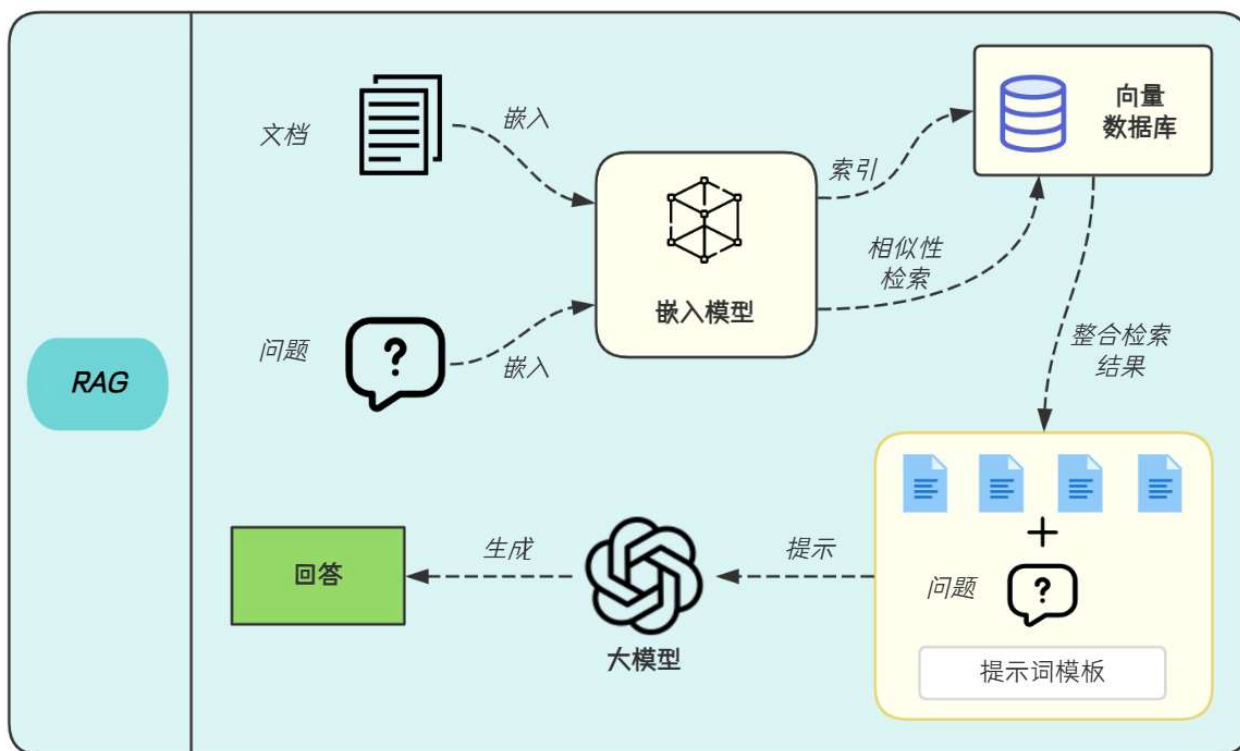


图 4.11 RAG 具体流程图

从上图 4.11 来看，RAG 的整个流程无非在于检索和生成，在上诉两个章节已经完成了文档预处理、知识库构建，以初步为 RAG 流程建立准备。现在开始实现知识库的检索功能，首先从向量数据库中检索与提问内容最相似的文档块，根据 `top_k` 参数候选出若干个文档块，再根据检索规则筛选出 `k` 个文档块返回。最终再把返回的文档块糅合进提示词模板中，再次输入大模型中，大模型根据提示词引导输出回答用户。

为了测试检索的效果，本平台实现了一个检索文档的测试接口，通过传入查询内容、`k` 与 `top_k` 参数从向量数据库中检索出符合条件的文档块，每个文档块将携带对应的分数，该分数用于表示与查询内容语义上最为接近的评分分数。

以下是本平台知识库检索的具体流程：

- (1) 通过知识库 Id 获取对应的知识库实例。如果知识库不存直接返回提示。
- (2) 通过步骤 1 获取的知识库实例，调用知识库搜索方法。
- (3) 根据不同知识库类型调用对应向量数据库的底层检索方法。
- (4) 底层检索方法离不开两个流程：①向量化提问内容、②传入向量以供检索。
- (5) 向量数据库根据检索参数（k 与 top_k）返回检索结果，检索结果以文档块列表的形式返回，同时附带每个文档块对应分数。

```
def do_search(self, query: str, k: int, fetch_k: int, **kwargs) -> list[tuple[Document, float]]:
    return self.vectorstore.similarity_search_with_relevance_scores(
        query=query,
        k=k,
        fetch_k=fetch_k,
        **kwargs
    )
```

图 4.12 知识库搜索方法

```
@router.post(path: '/search_test/{id}', summary='检索文档测试')
def search_test(
    kb_id: int = Path(default: ..., alias="id", description="知识库id"),
    query: str = Body(default: ..., description="查询内容"),
    k: int = Body(default: ..., description="返回前k个结果"),
    fetch_k: int = Body(default: ..., description="返回前fetch_k个结果"),
):
    kb = KBServiceFactory.get_service_by_id(kb_id)
    if not kb:
        return APIResponse(message="知识库不存在")
    document_with_score = kb.search(
        query=query,
        k=k,
        fetch_k=fetch_k,
    )
    document_with_score = [
        {
            "text": doc.page_content,
            "source": os.path.basename(doc.metadata.get("source", "")),
            "score": f"{score:.4f}"
        }
        for doc, score in document_with_score
    ]
    return APIResponse(data=document_with_score)
```

图 4.13 知识库检索测试接口

现在已经完成了 RAG 的检索步骤，还需大模型生成总结步骤。为了保证知识库模块的功能独立性且职责明确，RAG 剩余步骤将在下文聊天模块体现。

4.2 聊天应用模块

本小节将讨论大模型聊天应用的实现，将使用 LangChain 提供的聊天模型 API 构建。从基础单轮对话到多轮携带聊天历史对话，以及利用 Agent 实现工具调用，并且响应支持流式输出（打字机效果）。最后对 RAG 剩余步骤（大模型总结）利用工具调用的方式进行实现。

4.2.1 聊天模型工厂

对于日新月异的大语言模型，为及时支持聊天模型的接入，该模块使用工厂设计模式编码实现，通过传入聊天模型的名称即可得到模型实例，实现开箱即用。

具体实现代码如图 4.14 所示：

```
class ChatModelFactory:
    4 usages  1 shuofeng *
    @staticmethod
    def get_chat_model(model_name: str, api_key: str = None, temperature: float = 0.5, max_tokens: int = None, model_config=None) -> BaseChatModel:
        """
        根据模型名称获取聊天模型
        """
        if not model_name:
            raise BusinessException("模型名称不能为空")
        if not api_key:
            api_key = get_llm_api_key(model_name)
            if not api_key:
                raise BusinessException(f"未找到模型对应的api_key: {model_name}")
        llm_list = get_supported_llm_list()
        platform_name = ""
        for llm in llm_list:
            llms = list(llm.values())[0]
            if model_name in llms:
                platform_name = list(llm.keys())[0]
                break
        if model_config is None:
            model_config = get_llm_config(platform_name)
        if not platform_name:
            raise BusinessException(f"未找到模型对应平台: {model_name}")
        elif platform_name == "zhhipuai":
            from langchain_community.chat_models import ChatZhipuAI
            return ChatZhipuAI(
                model=model_name,
                api_key=api_key,
                temperature=temperature,
                max_tokens=max_tokens,
                **model_config
            )
        elif platform_name == "deepseek":
            from langchain_deepseek import ChatDeepSeek
            return ChatDeepSeek(
                model=model_name,
                temperature=temperature,
                max_tokens=max_tokens,
                api_key=api_key,
            )
        else:
            raise BusinessException(f"不支持的模型: {model_name}")
```

图 4.14 聊天模型工厂代码实现

4.2.2 多轮对话记忆聊天应用设计

大型语言模型是一种先进的机器学习模型，在文本生成、翻译、问答等各种与语言相关的任务中表现出色，而无需针对每种场景进行特定于任务的微调。在 LangChain 提供了聊天模型的概念，通过聊天模型的接口进行访问以消息列表作为输入，模型将会返回一条消息作为输出。消息作为聊天模型的通信单位，用于表示聊天模型的输入和输出，一条消息包括角色、内容、附加元数据，通常以列表的形式出现，根据角色可分类为：系统消息、用户消息、助手消息、工具消息。当然消息的内容也存在多种格式，对于多数聊天模型一般为文本内容，对于支持多模态的模型可以为图像、音频等格式。

提示模板负责将用户输入格式化为可以传递给语言模型的格式（也就是转换为字符串或消息列表），这可以用来指导模型的响应，帮助其理解上下文并生成相关且连贯的语言输出。为了引导聊天模型的行为，一般指定模型采用特点的角色或设定对话的背景基调（例如，你是一位软件工程博士），确保任务执行者能从专业的视角出发处理问题^[11]。所以创建提示最自然的方式是基于人类手动创建直观的模板^[12]。在 LangChain 在调用聊天模型时通常会使用 `SystemMessage`（系统消息）作为指令为模型引导输出的内容，以达到预取的输出效果。

用户的提问信息一般会以 `HumanMessage` 的形式输入给模型。而对于 `ToolMessage`（工具消息）是处理聊天模型调用某个外部工具（API、文件）产生的消息内容。`ToolMessage` 用于实现模型与外部环境交互而产生的输入模式，通常工具消息用于承载模型想要调用的外部资源的输入或外部资源响应的结果，模型通过 `ToolMessage` 的内容间接的与外部系统进行交互，达到了“智能体”的初步效果。

正是因为消息列表的概念，模型的输入可以是多条消息，对于进行过多轮对话的模型，再下一次的对话时传入聊天历史，可以让模型“回忆”之前的交互信息。利用这个特点可以设计出拥有多轮聊天记录聊天机器人。

在 LangChain 中具有许多不同提供商的聊天模型集成。这些模型通常分为两个类型：①官方模型、②社区模型。这些聊天模型都实现了 `BaseChatModel` 接口，所有的模型天生拥有异步调用、流式输出、批处理和结构化输出等功能，同时许多关键方法都使用消息作为输入，同样地，模型以消息作为输出。聊天模型提供了一组基本参数，用于配置控制模型行为，例如：温度、最大令牌数、以及等待响应时间。

对于本平台而言，设计一个聊天应用只需要借助 LangChain 开箱即用的 API 即可完成搭

建。以下是本平台实现的拥有多轮对话记忆的聊天应用具体代码，如图 4.15 所示：

```
@router.post(path: '/completions/debug', summary='大模型聊天debug接口')
async def chat_debug(
    query: str = Body(default: "", description="用户提问"),
    messages: list[History] = Body(default: [], description="消息列表"),
    model: str = Body(default: "glm-4", description="模型名称"),
    max_tokens: int = Body(default: 1024, alias="maxTokens", description="最大输出长度"),
    temperature: float = Body(default: 0.5, description="温度"),
    stream: bool = Body(default: True, description="流式输出"),
):
    # 记录链路日志
    set_langsmith_env()
    # 拟输入的对话消息
    input_messages = []
    # 通过聊天模型工厂类获取聊天模型对象
    chat_model = ChatModelFactory.get_chat_model(
        model_name=model,
        temperature=temperature,
        max_tokens=max_tokens
    )
    # 如果消息列表为空且用户提问不为空，则将用户提问作为本次历史消息
    if not messages:
        if query:
            messages.append(History(role="user", content=query))
        else:
            return APIResponse.fail(message="提问为空")
    input_messages.extend([message.to_msg_tuple() for message in messages])
    logger.info(
        f"\nmessages: {input_messages}\nmodel: {model}\nmax_tokens: {max_tokens}\ntemperature: {temperature}\nstream: {stream}"
    )

    if stream:
        streaming = chat_model.astream(input=input_messages)
        return EventSourceResponse(_common_sse_iterator(streaming))
    else:
        result = await chat_model.ainvoke(input=input_messages)
        return APIResponse(data=result)
```

图 4.15 聊天接口实现细节

4.2.3 基于 Agent 工具调用具体实现

设计出多轮对话记忆的聊天应用之后，将开始解决模型如何调用外部系统的问题。众所周知，聊天模型本身无法采取行动，它们只是单纯的输出文本内容，但如果要求模型输出指令，并交由某个执行人手中代为实施和运行，最后将执行结果返回给模型，模型读取执行结果来推理出最终的结果，这是可行的。

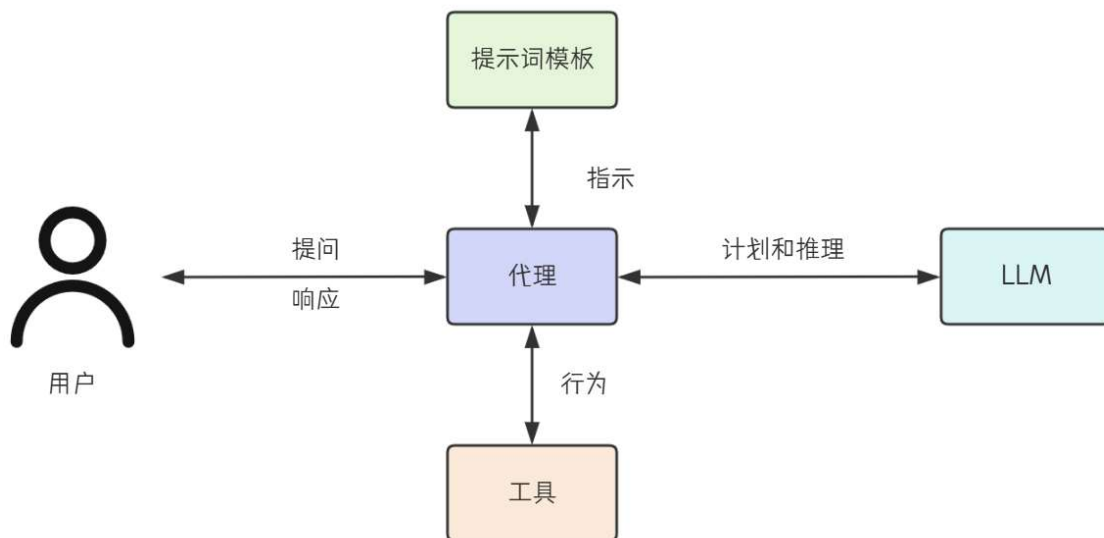


图 4.16 Agent 原理示意图

如图 4.16 所示从整个过程来看，模型只是作为推理引擎来决定采取哪些行动以及这些行动的依据（输入）是什么，具体的执行者称为代理，它负责将行动的转换为具体的实施，同时将行动结果反馈给模型。得到反馈的模型再次进行推理确定是否达到预取结果或者需要更多操作，循环往复直到完成闭环，从用户角度来看像是模型完成了对于外部系统的访问。总的来说，代理的核心思想是使用模型来选择一系列要采取的行动。

对于构建具有智能的聊天应用，在 LangChain 中使用 AgentExecutor 作为代理运行时，正如上文所述，代理可以帮助模型调用工具采取一系列行动。Agent 实现具体步骤：①定义工具、②创建代理、③添加记忆、④运行代理。

（1）定义工具。

LangChain 支持从函数、Runnable 对象、继承 BaseTool 的方式来定义一个工具。本平台使用自定义 BaseTool 基类（CustomTools）方式实现工具定义，具体工具只需要实现该类即可被模型调用。最后实例化对应工具，注册进工具管理器中保证项目中工具对象是单例的。

工具管理器支持通过传入工具 id 和工具名称获得对应的工具实例，也可获取平台中所有注册可用的工具列表。当创建聊天应用后，聊天应用通过关联工具 id 的方式，再每一次调用模型时根据 id 得到工具列表并与模型进行绑定，后续可被代理运行时调用执行。

具体工具管理器如图 4.17 所示。

```
class ToolsManager(ABC):
    _tools_instances: dict[int, CustomTools]
    _tools_info_map: dict[int, ToolInfo]

    @shuofeng
    def __init__(self):
        self._tools_instances = {}
        self._tools_info_map = {}

    @shuofeng
    def register_tool(self, base_tool: CustomTools):
        """
        注册tool实例
        """
        id = base_tool.tool_id
        if not id:
            raise BusinessException(f"id can not be null")
        if id in self._tools_instances:
            raise BusinessException(f"Tool with id {id} is already registered.")
        self._tools_instances[id] = base_tool
        log.info(f"tool:{id} register success", id)
        self._tools_info_map[id] = ToolInfo(
            tool_id=id,
            tool_name=base_tool.get_tool_name(),
            tool_description=base_tool.get_tool_description(),
            tool_args_schema=base_tool.get_args_schema()
        )

    @shuofeng
    def get_tool_by_id(self, id: int) -> CustomTools:
        """
        根据id获取已注册的Tool实例
        """
        ins = self._tools_instances.get(id, None)
        if not ins:
            raise BusinessException(f"tool[{id}] is not found")
        return ins

    @shuofeng
    def get_tools_by_ids(self, ids: list[int]) -> list[CustomTools]:
        """
        根据ids获取已注册的Tool实例列表
        """
        return [self._tools_instances.get(id, None) for id in ids]

    @shuofeng
    def get_tools(self) -> list[CustomTools]:
        """
        获取所有已注册的Tool实例
        """
        return list(self._tools_instances.values())
```

图 4.17 工具管理器定义图

(2) 创建代理。

接着是创建代理，模型在输出时是需要继续推演，相当于人在计算时需要一张草稿纸。所以在输入消息时，借助 LangChain 提供的 MessagesPlaceholder 消息占位符机制，将模型的

推演结果暂存至占位符之中。需要注意的是 LangChain 代理只负责接受输入并决定要采取的行动，但是具体的操作并不会由它完成，而是交给 AgentExecutor 完成。对于代理执行的具体操作原理是 LangChain 定义的工具调用概念，其内部具体执行流程如图 4.18 所示：

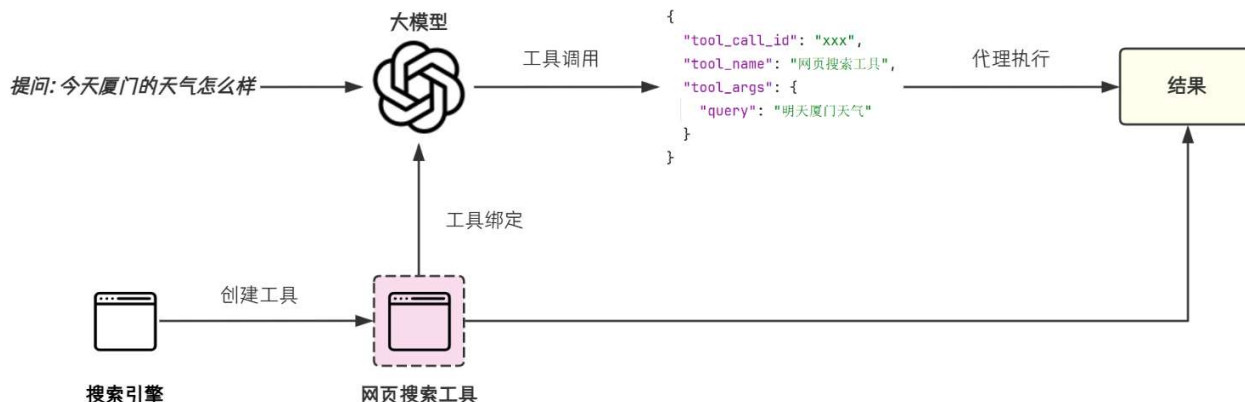


图 4.18 工具调用内部具体流程

(3) 添加记忆。

由于代理是无状态的，这意味着它不记得之前的聊天记录，为了赋予其记忆同样利用消息占位符的机制，将当前会话的多轮聊天消息传入即可。

(4) 运行代理。

当代理开始被调用运行时，模型会理解用户提问的问题场景，读取提供的工具列表尝试利用工具解决问题，推理出工具的输入参数，由代理运行时去调用执行，并以工具消息的方式通知模型，模型根据整个上下文从工具消息获取对用户问题相关的部分信息，最后提炼总结输出。

4.2.4 知识库工具实现全流程 RAG

从上文所述 RAG 流程来看，关键点在于结合检索（从知识库中获取相关信息）和生成（基于检索结果生成回答）两个步骤。在上文已经通过引入知识库的方式解决了检索问题，现在还剩生成步骤还未实现。

由于生成步骤与大模型紧密结合，同时本模块已实现了聊天应用的构建，所以将 RAG 的生成步骤绑定在聊天应用中更加符合直觉。在聊天应用中，为了提升模型回答的准确性，需要引入外部知识库作为参考，具体步骤：将检索到的内容与用户提问拼接成提示词，输入提示词以指导模型生成。对于聊天模型而言，知识库属于外部系统，模型无法直接从知识库得到任何参考。

为此，能否将知识库看作一种工具，模型通过调用知识库工具的方式实现检索。

答案是可行的，在上一小节中已验证基于 Agent 可实现模型对外部系统的访问，只需要将知识库包装成工具，模型推理是否要调用知识库工具且生成对应的工具参数，接着由代理介入实际调用知识库进行检索，得到检索结果并与上下文组装，再次输入模型，以指导模型总结生成。

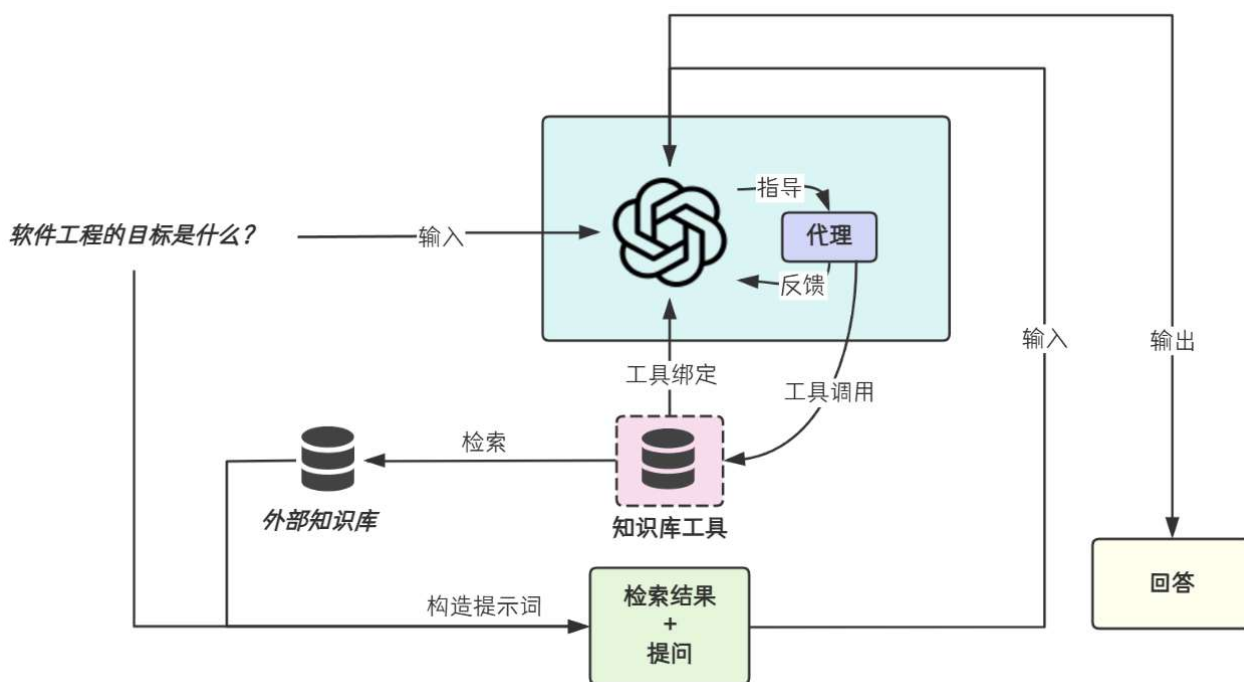


图 4.19 知识库工具调用

从图 4.19 整个流程来看，符合 RAG 的生成步骤，也就是基于检索结果生成回答。

以下是知识库工具的代码具体实现，如图 4.20 和图 4.21 所示：

```
def _run(self, query: str, kb_ids: list[int], k: int, fetch_k: int, score_threshold: float) -> str:
    docs_with_score = []
    try:
        for kb_id in kb_ids:
            # 根据kb_id得到kbService
            kb_service = KBServiceFactory.get_service_by_id(kb_id)
            if not kb_service:
                raise ToolException(f"知识库[{kb_id}]不存在")
            # 检索得到docs
            docs_result = kb_service.search(query=query, k=k, fetch_k=fetch_k, score_threshold=score_threshold)
            if not docs_result:
                continue
            docs_with_score.extend(docs_result)
        return self.docs_cast_to_prompt(docs_with_score)
    except Exception as e:
        raise ToolException(e)
```

图 4.20 知识库工具代码实现 1

```
def docs_cast_to_prompt(self, docs: list[tuple[Document, float]]) -> str:
    if not docs:
        raise ToolException("No documents found.")
    docs_content = ""
    prompt_template = PromptTemplate.from_template(
        "# 检索结果\n"
        "根据【知识库工具】检索，从知识库中检索出{len}条文档知识块，并且按照与问题相关度的倒叙进行排序。\\n"
        "以下是检索出的文档块内容：\\n\\n"
        "{docs_content}"
    )
    # 构造文档内容
    for index, item, in enumerate(docs):
        doc = item[0]
        score = item[1]
        docs_content += (f"## 文档块{index + 1}-{doc.metadata.get('header', '')}\\n"
                        f"得分：{score}\\n"
                        f"来源：{doc.metadata.get('source', '暂无')}\\n"
                        f"文件id：{doc.metadata.get('source_file_id', 0)}\\n"
                        f"文件名：{doc.metadata.get('source_file_name', '')}\\n"
                        f"内容：{doc.page_content}\\n\\n")
    # 组装prompt
    prompt_val = prompt_template.invoke(
        {
            "len": len(docs),
            "docs_content": docs_content,
        }
    )
    return prompt_val.to_string()
```

图 4.21 知识库工具代码实现 2

4.3 本章小结

本章对本平台涉及的核心模块进行详细的叙述，从设计和实现的角度出发，阐述了核心模块的实现细节。

第五章 平台实现

5.1 知识库实现

知识库是平台中用于存储教学文档的知识集合，由结构化的数据组成，如：Docx、PDF、Markdown 文件。用户可用针对相同主题的文档建立独立的知识库，使用对应的文档加载器读取上传的文档并使用合适的分词器将内容拆分为文档块以供检索。

5.1.1 知识库列表页面

在知识库列表页面展示用户创建的所有知识库，可通过页面右上角新建按钮创建新的知识库。创建知识库可命名，支持选择嵌入模块和存储的基础设施向量数据库。

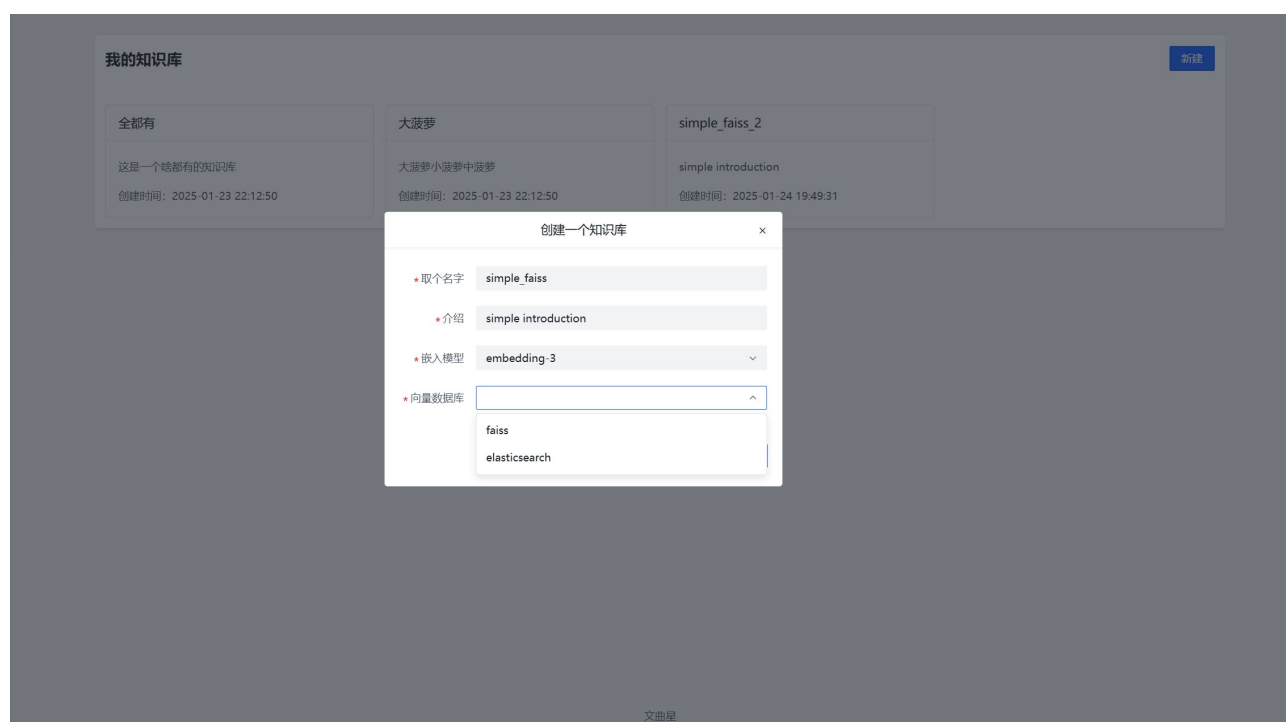


图 5.1 知识库列表页面

5.1.2 知识库详情页面

在详情页面，用户可对建立的知识库进行导入文档、管理文档、检索操作等操作。

(1) 数据集：可预览所有已上传的文档文件，并且对文档进行删改查操作。



图 5.2 知识库详情页-数据集

(2) 导入数据: 支持直接上传文档, 并选择对应的文档加载器和分割器, 最后自动分块。

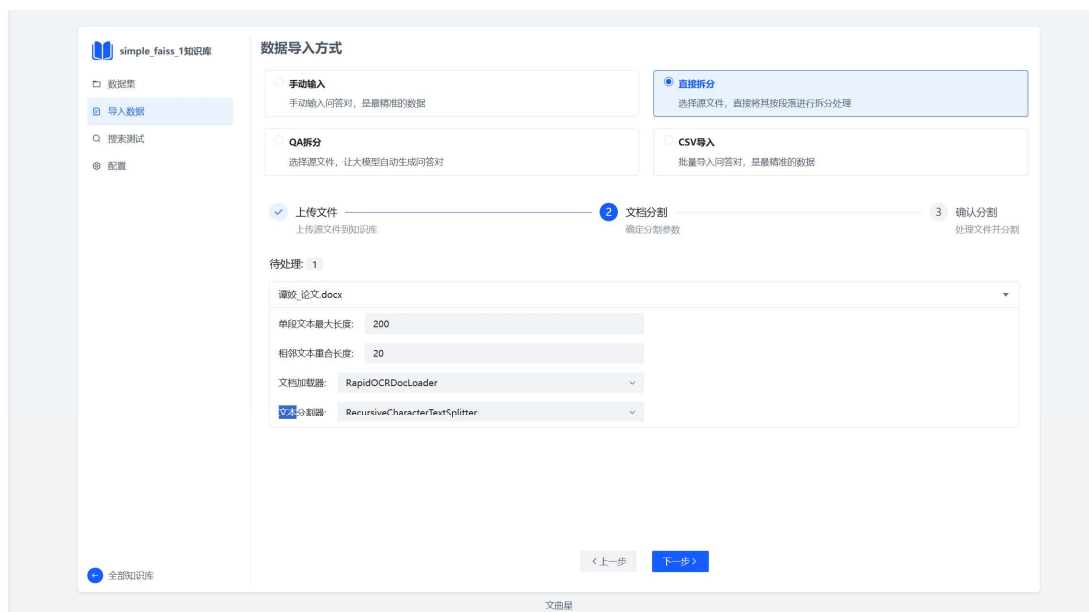


图 5.3 知识库详情页-导入数据

(3) 搜索测试: 用于测试文档拆分效果是否可被检索到, 可设定检索参数。



图 5.4 知识库详情页-搜索测试

(4) 配置：支持知识库信息进行修改操作。

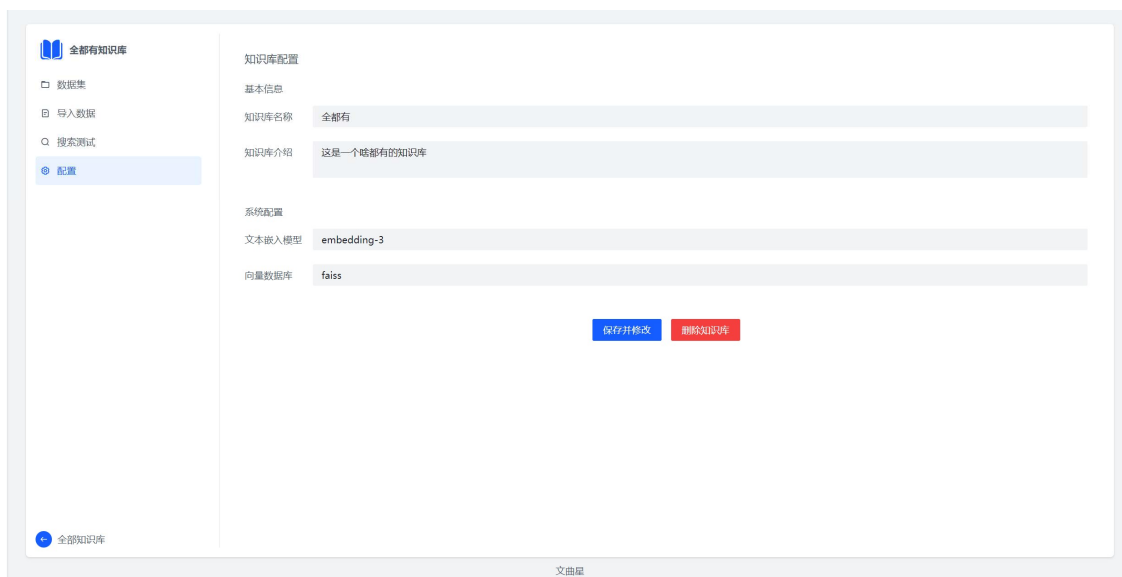


图 5.5 知识库详情页-配置

5.2 聊天应用实现

聊天应用是由用户自主创建的 AI 对话定制应用，用户可引导聊天模型的输出内容，具体表现为可通过编写系统提示词，精准引导聊天模型的输出风格和内容。用户可为聊天模型添加外部工具，模型可根据用户提问内容自动调用最适合的工具完成任务，实现智能体效果。

同时也可集成知识库功能，用户提问的过程中，聊天模型在每一次回答前都尝试从知识库获取答案，最后归纳总结确保回答的准确性。

5.2.1 应用列表页面

在应用列表页面可查看用户创建的所有聊天应用，同时也可根据应用发布状态进行筛选。也可创建全新的聊天应用，并支持从模板创建。

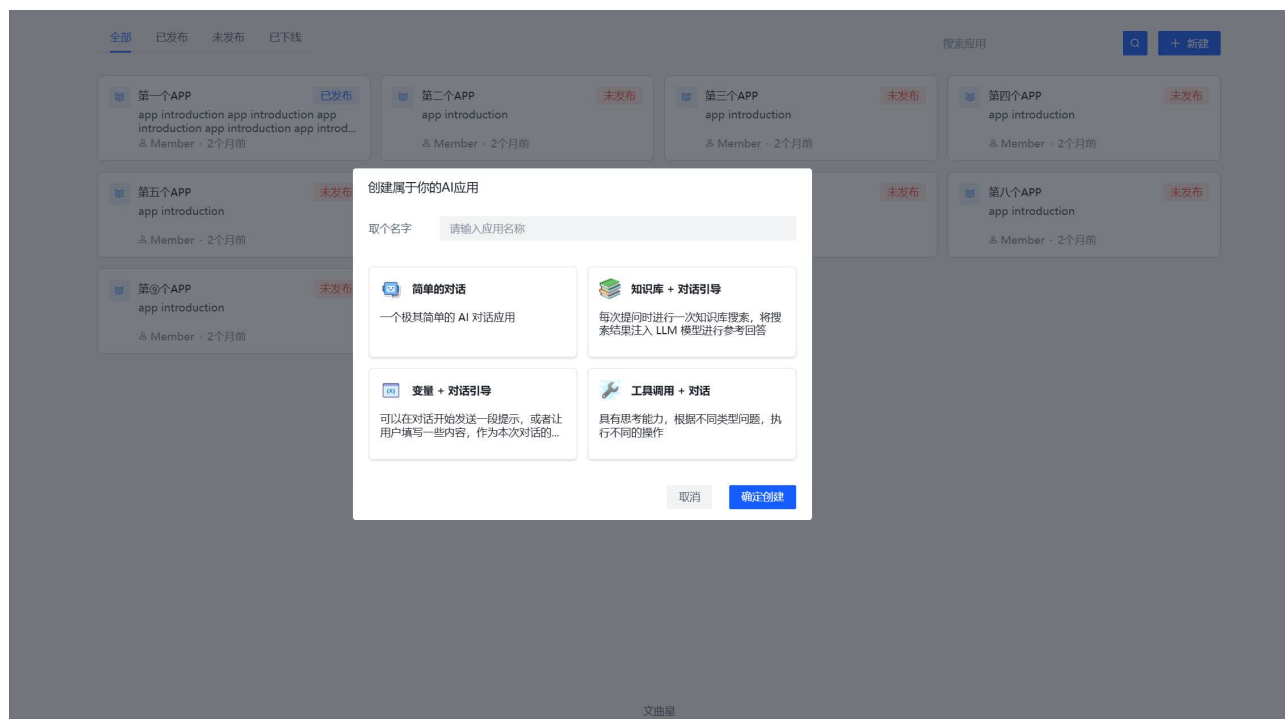


图 5.6 全部应用列表

5.2.2 应用详情页面

点击某个应用进入详情页面，可修改应用的基本信息，也可调节聊天模型的常用参数包括：模型型号、温度、最大回复 token 数、系统提示词等。同时支持高级配置包括：关联知识库、工具调用等，提供高度定制化提升对话体验。

(1) 简易配置：支持配置应用的基本信息和模型的基础参数。

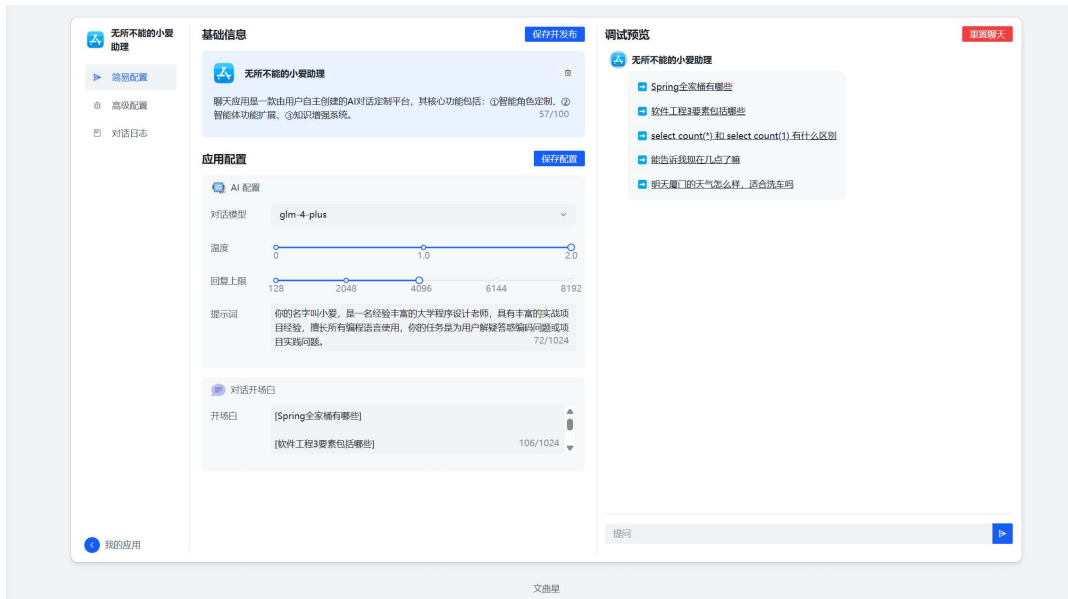


图 5.7 应用详情页-简易配置

(2) 高级配置：扩展模型能力，支持工具调用和绑定知识库。

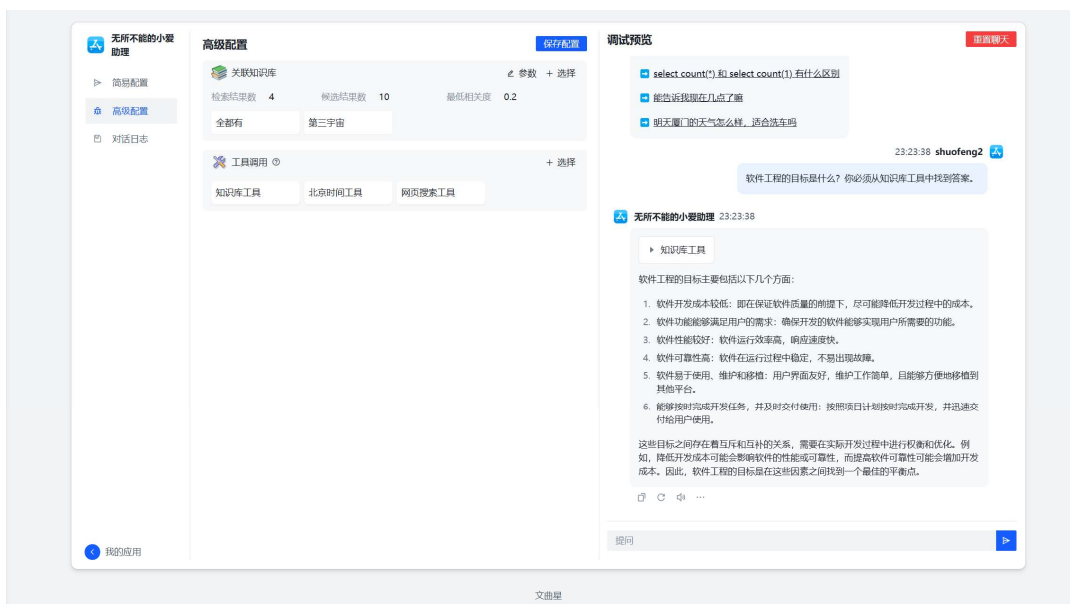


图 5.8 应用详情页-高级配置

(3) 关联群组：设定聊天应用关联的群组，以便于发布至用户组中。

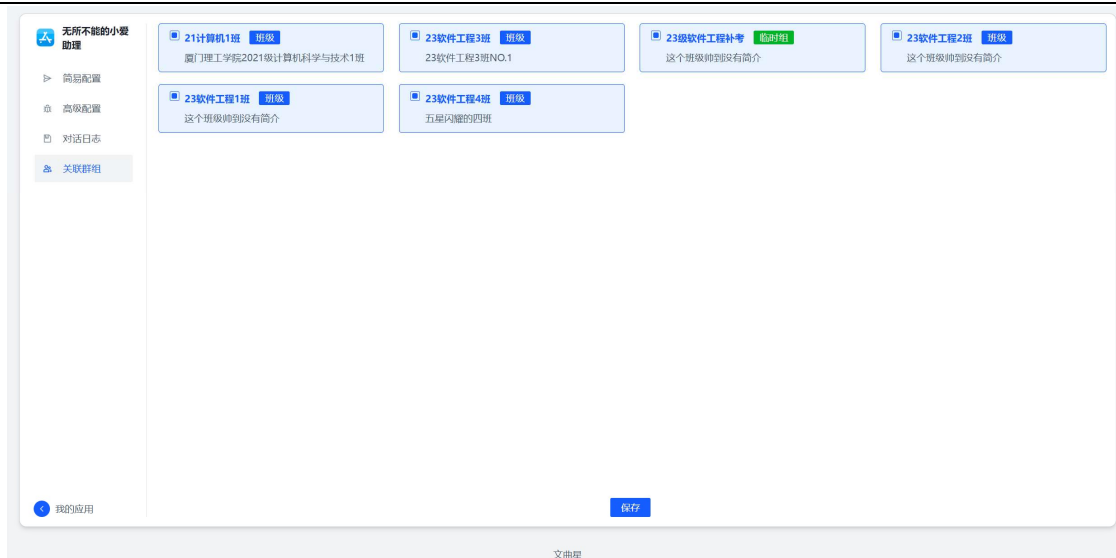


图 5.9 应用详情页-关联群组

5.2.3 聊天流程演示

本次聊天流程将演示模型的多轮记忆情况、工具调用、角色扮演场景出发。

- (1) 角色扮演：首先告诉模型用户名字再询问其角色名字。
- (2) 工具调用：接着尝试提问当前时间，对于模型而言其不知道实时时间，应该尝试调用工具。
- (3) 工具调用：然后再尝试询问模型实时更新的问题，考验其检索能力。
- (4) 推理能力：提问专业知识提示其使用知识库工具寻找答案解决问题。
- (5) 聊天历史：多轮对话后，询问第一轮对话的事实。

经测试，本平台开发的聊天模型应用完美实现了上述预设的多个场景，具体效果如图 5.10 所示：

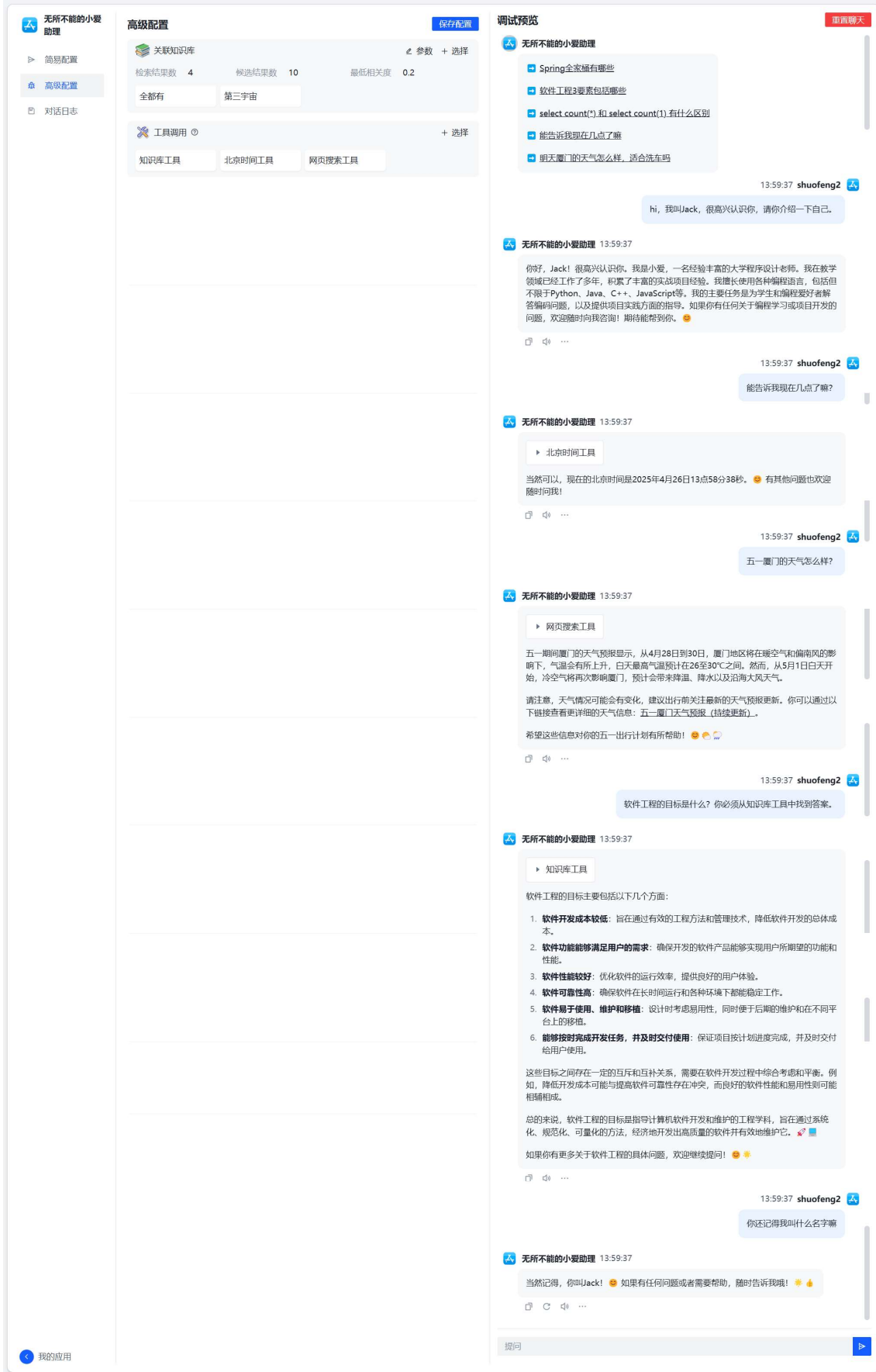


图 5.10 聊天流程测试

5.3 本章小结

本章节从平台实现的平台出发，着重介绍了平台涉及的知识库模块和聊天应用模块，从多个角度出发详细说明了实现的细节。其中，对于聊天应用模块进行初步功能测试，同样从设计的初衷出发，考验了聊天模型实现的多种能力。

第六章 系统测试

6.1 接口测试

6.1.1 前期准备

为了确保本平台的稳定性和可靠性，将对平台涉及的接口进行自动化测试。为了防止测试数据出现差异以下统一测试环境，测试环境配置如表 6.1 所示：

表 6.1 测试环境说明表

环境	值
操作系统	Windows 11
运行内存	32G
硬盘	1T
处理器	AMD Ryzen 7 7840HS

由于本平台采用前后端分离的思想，前端通过接口统一访问后端的微服务集群换取数据，所以接口的性能是本平台综合性能的直接影响。当每个模块对应的前端页面和后端接口开发完毕之后，为了进行联调操作，后端以 API 文档的方式向前端提供联调手段。本平台使用 Apifox 作为接口调试工具，同时 Apifox 也具备完善的接口自动测试功能，总之为了减少手动编写接口测试脚本的成本，故采用 Apifox 作为接口自动化测试工具。

首先，添加测试步骤，将联调期间的接口导入作为测试步骤。



图 6.1 导入接口弹窗

接着设置自动化测试的循环次数和并发使用的线程数，这里为了验证接口的可用性，不考虑并发情况，所以将线程数置为 1。



图 6.2 自动化测试参数设置

6.1.2 测试结果



图 6.3 接口自动化测试结果

从图 6.3 测试结果来看，共执行了 41 个测试用例，所有测试用例均通过，可得出结论：

①接口功能表现良好：自动化测试覆盖的所有接口均能正常响应并返回预期结果，说明这些接口的基本功能是正常的。②响应性能整体可接受：大部分接口的响应时间都在 50ms 之内，总耗时和平均耗时也处于较低水平。例如：具体某些接口（如“上传知识库文件”耗时 507ms，“创建知识库”耗时 316ms，“添加文档块”耗时 3.85s）的响应时间相对长一些，这些操作涉及与第三方 API 调用并发数受限于第三方，但仍在可接受范围内，用户体验良好。③测试覆盖范围较广：测试用例覆盖了平台的多个核心模块，包括用户登录、知识库的增删改查、文件上传、模型调用、应用配置、对话历史等表明测试较为全面。

6.2 功能测试

本节将针对该平台开发实现的较为主要关键的模块功能进行一系列测试。在测试过程中采用常见的黑盒测试方法，该方法与白盒测试相比主要的关注点在于系统的表现行为。本次功能测试选取以下核心功能点：①登录注册、②知识库操作、③检索测试、④聊天应用操作、⑤调试聊天应用。下面将对以上涉及的功能点设计并编写对应的测试用例，以保证系统在各角度符合预期操作。

（1）登录注册测试用例

表 6.2 登录注册用例

用例编号	预设条件	测试步骤	预期结果	实际结果
1	数据库不存在手机号为 17123456789 的用户。	访问注册页，正确输入手机号为 17123456789 和正确密码位数进行注册操作。	平台成功通过注册，跳转至登录页。	符合预期结果正常执行。
2	数据库存在手机号为 17123456789 的用户。	访问注册页，输入预设手机号和正确密码位数进行注册操作。	平台阻止用户注册，提示手机号已存在。	符合预期结果正常执行。

续表 6.2

用例编号	预设条件	测试步骤	预期结果	实际结果
3	数据库存在手机号为 17123456789 和密码为 123 的用户。	访问登录页，输入预设的手机号和密码点击登录。	平台通过校验，根据角色跳转至对应的首页。	符合预期结果 正常执行。
4	同用例 3。	访问登录页，输入预设的手机号和密码 321，点击登录。	平台验证失败，提示拒绝登录。	符合预期结果 正常执行。

以上登录测试用例分别按照正面与反面出发测试，从测试结果来看，平台系统能正确处理不同的测试数据，与编码逻辑一致。

（2）知识库操作

表 6.3 知识库操作用例

用例编号	预设条件	测试步骤	预期结果	实际结果
1	不存在名为 kb_test 的知识库。	新建名为 kb_test 的知识库。	成功创建知识库并跳转至详情页。	符合预期结果 正常执行。
2	已存在名为 kb_test 的知识库。	新建名为 kb_test 的知识库。	平台检测到同名知识库阻止创建。	符合预期结果 正常执行。
3	已存在名为 kb_test 的知识库。	进入 kb_test 详情页删除知识库。	平台删除知识库自身，并删除已上传的文档，最后跳转返回知识库列表。	符合预期结果 正常执行。
4	已存在名为 kb_test 的知识库。	修改知识库名称为 kb_test_modify 和知识库介绍。	提示知识库名称修改成功，并跳转至知识库列表。	符合预期结果 正常执行。
5	已存在名为 kb_test 的知识库。	上传 Markdown 文档到知识库但不分割文档。	文档上传成功，可查询到文档，但暂无拆分的文档块数据。	符合预期结果 正常执行。

续表 6.3

用例编号	预设条件	测试步骤	预期结果	实际结果
6	已存在名为 kb_test 的知识库并上传名为 test_doc.md 的文档文件。	上传与 test_doc.md 同名文档。	文件上传失败，提示存在同名的文档。	符合预期结果 正常执行。
7	已存在名为 kb_test 的知识库且无上传任何文档。	上传 test_doc.md 文档到知识库并选对应文档加载器和分离器。	文档上传成功，并且成功拆分出文档块。	符合预期结果 正常执行。

本用例设计了对知识库可进行的基本操作，从基础的新增知识库到删除知识库，以及知识库中的上传和分割文档操作，根据表中预期结果来看，符合编码逻辑，证明知识库模块功能操作可靠。

(3) 检索测试

表 6.4 知识库检索用例

用例编号	预设条件	测试步骤	预期结果	实际结果
1	存在已上传并分割的文档数据。	检索与文档相关的数据并设置检索参数 k 为 2 和 fetch_k 为 10	检索结果返回 2 条数据，并按照相关性分数排序。	符合预期结果 正常执行。
2	存在已上传并分割的文档数据	检索与文档相关的数据并设置检索参数 k、fetch_k 都为 5	检索结果返回 5 条相关数据，并按照相关性分数排序。	符合预期结果 正常执行。
3	存在已上传并分割的文档数据	检索与文档毫无相关的数据并设置检索参数 k、fetch_k 都为 10	检索结果返回 10 条不相关数据，并且相关性分数极低。	符合预期结果 正常执行。

该用例针对知识库中已上传并且拆分之后的文档块进行搜索测试，需要判断检索的内容

与检索的结果是否相关联。比如：检索 [线程池] 关键字，检索结果需要包含 [线程池] 相关语义的文档块，并且按照语义相关度进行排序。经过测试所有用例均通过，符合编码逻辑。

(4) 聊天应用操作

表 6.5 聊天引用操作用例

用例编号	预设条件	测试步骤	预期结果	实际结果
1	空的聊天应用列表。	新建名为 <code>app_test</code> 的聊天应用	提示创建成功，并且应用列表出现新建的应用状态为未发布。	符合预期结果 正常执行。
2	已存在名为 <code>app_test</code> 的应用。	修改 <code>app_test</code> 的名称为 <code>app_rename</code> 并且修改应用简介，最后保存并发布。	提示发布成功，数据持久化到后端。	符合预期结果 正常执行。
3	已存在名为 <code>app_test</code> 的应用。	修改 <code>app_test</code> 应用配置，选择模型、调整温度、最大输出 token 数、提示词等参数，并保存配置。	提示保存成功，数据持久化到后端。	符合预期结果 正常执行。
4	已存在名为 <code>app_test</code> 的应用。	自定义应用对话开场白配置并保存，查看调试预览是否正常显示开场白。	提示保存成功，并且调试预览出现开场白聊天气泡。	符合预期结果 正常执行。
5	已存在名为 <code>app_test</code> 的应用，并且无关联任何群组。	全选已存在的群组并保存配置。	提示关联成功，并且在用户已加入的群组中正确显示 <code>app_test</code> 聊天应用。	符合预期结果 正常执行。
6	已存在名为 <code>app_test</code> 的应用。	从配置页中点击删除本应用按钮。	提示删除成功，并且跳转应用列表。	符合预期结果 正常执行。

聊天应用是本平台中最为关键的模块，确保该模块操作的合理性和稳定性是重中之重。以上用例列举了用户可进行的所有基本操作，通过逐一测试将实际结果与预期行为对比，发现无任何差错符合设计逻辑。

（5）调试聊天应用

表 6.6 调试聊天应用用例

用例编号	预设条件	测试步骤	预期结果	实际结果
1	已存在名为 app_test 的应用,设置系统提示词【你的名字叫小爱】。	进行多轮对话，对话内容如：①用户的昵称叫 Jack。②提问应用预设角色昵称。③询问用户的昵称，并正确回答。	应用通过多轮次对话后，仍然可回忆与用户的多轮聊天记录。	符合预期结果 正常执行。
2	已存在名为 app_test 的应用,并且已设置基本 AI 参数，开场白配置[软件工程三要素]、[软件工程的目标]两句话。	调试预览显示开场白对话气泡中预设的两句内容，点击任意一句开场白，作为用户输入发送开场白内容提问聊天应用。	正确显示开场白内容，当用户点击开场白时作为提问内容发送给应用，应用由聊天模型驱动进行对话输出。	符合预期结果 正常执行。
3	已存在名为 app_test 的应用,并且设置模型最大输出 token 数为 128 和温度为 0。	在调试预览页面输入框中向应用提问【LLM 是什么】	正确回答用户提问并且以打字机的形式输出，但回答被截断即回答不完整。	符合预期结果 正常执行。
4	保留用例 2 预设条件和测试结果。	在调试预览页面再次提问相同问题【LLM 是什么】	正确回答用户问题，但回答内容不完整且与用例 2 的回答高度一致。	符合预期结果 正常执行。

续表 6.6

用例编号	预设条件	测试步骤	预期结果	实际结果
4	已存在名为 <code>app_test</code> 的应用并已配置基本参数,关联已存在的知识库。	配置工具调用, 选择【知识库工具】并且提问知识库中存在的相关数据。	应用成功调用【知识库工具】检索用户提问的相关内容, 最后从检索结果中提炼总结回答。	符合预期结果 正常执行。
5	已存在名为 <code>app_test</code> 的应用并已配置基本参数。	配置工具调用, 选择【北京时间工具】并询问聊天应用当前时间。	应用成功调用【北京时间工具】获取当前时间, 最后格式化工具提供的时间为直观形式回答用户。	符合预期结果 正常执行。
6	已存在名为 <code>app_test</code> 的应用且配置基本参数,也配置【知识库工具】调用。	配置工具调用, 选择【网页搜索工具】,并提问应用厦门明天天气状况。	应用成功调用【网页搜索工具】获取厦门城市天气信息, 根据提问内容提炼总结回答。	符合预期结果 正常执行。

本用例通过调试方式进行测试, 覆盖了聊天应用的所有使用场景, 从基本的 AI 参数配置、开场白设定出发, 到高级配置关联知识库检索, 使用外部工具给应用中模型附加获取外部数据的能力, 均完整实现且符合预期功能设计。

6.3 本章小结

本章主要对系统平台核心模块进行软件测试, 从两个方面出发: ①接口自动化测、②功能测试。通过接口自动化测试, 确保系统前后端链路打通, 数据交互无异常。功能测试同样以核心模块为重点, 编写用户可进行的一系列操作, 模拟用户行为与预期结果对比确保平台功能点稳定, 符合设计初衷。对于微服务架构, 通常进行性能测试但由于本平台系统核心功

能点大多数依赖第三方接口，使得并发数被外部接口限制，但对日常操作无任何阻碍性，整体流程稳定且流畅。总结来说，本平台系统模块功能都能满足预期要求，拥有较好用户体验。

第七章 总结与展望

7.1 总结

本研究针对程序设计教学中存在的教学模式单一、个性化不足等问题，设计并开发了基于 LangChain 与大语言模型的智能教学辅助平台，取得以下创新性成果：

在系统架构方面，采用微服务架构设计，整合 Spring Cloud 与 FastAPI 框架。通过 Nacos 实现服务注册与配置管理，Gateway 作为统一 API 网关，结合 OpenFeign 的远程调用能力，有效克服传统单体架构的扩展问题。

知识库构建方面，创新性地采用 LangChain 的 UnstructuredFileLoader 处理多格式文档，设计中文递归字符文本分割器进行文档预处理。通过 Embedding 技术将文本向量化后存储于 FAISS 与 Elasticsearch 向量数据库中，实现检索增强生成机制，弥补了大模型在专业领域的知识局限性和幻觉问题。

聊天应用模块采用工厂模式支持多种模型的快速接入，基于 Agent 机制实现工具调用与对话管理。通过自定义工具基类与消息占位符技术，构建具有记忆能力的智能聊天应用，为学生提供个性化的编程指导。

7.2 展望

功能扩展将着重于：①开发多模态交互接口，整合图像识别与语音处理技术，支持代码截图解析与语音聊天问答等多种交互方式。②构建智能评估体，结合静态代码分析与大语言模型，提供个性化学习路径建议。

技术优化方向包括：①研究高效 Embedding 模型与近似最近邻搜索算法，优化知识库的检索效率。②探索模型量化与剪枝技术，推进在移动端部署的能力。③尝试多模态模型的协同机制，整合代码生成、逻辑推理等专项模型的优势，跳转复杂教学场景的应对能力。

参考文献

- [1] 高洪皓,陈章进.人工智能赋能程序设计课程教学改革[J].计算机教育,2024,(07):41-43+48.DOI:10.16512/j.cnki.jsjy.2024.07.029.
- [2] 鞠小林,张艳梅,王皓晨,等.基于大语言模型辅助教学的 Python 编程课程教学探索[J].计算机教育,2024,(09):33-37.DOI:10.16512/j.cnki.jsjy.2024.09.024.
- [3] 窦洽.人工智能技术在计算机辅助教学中的应用研究[J].信息系统工程,2024,(09):48-51.
- [4] 李烨红,黄培泉,周诗婷,等.基于微服务架构的自动判卷系统设计与实现[J].现代信息科技,2024,8(11):101-105+110.DOI:10.19850/j.cnki.2096-4706.2024.11.020.
- [5] 张彧圣.基于 SpringCloud 的在线学习系统的设计与实现[D].华中科技大学,2022.DOI:10.27157/d.cnki.ghzku.2022.000169..
- [6] FastAPI 团队. FastAPI [EB/OL]. <https://fastapi.org.cn/features/>: “FastAPI 特性” 部分内容,[2024-12-15].
- [7] 张曦允,谭锴,欧阳涛辉,等.基于大模型的个性化习题推荐系统设计与实现[J].数字技术与应用,2024,42(7):32-34.
- [8] 高雅奇.基于大语言模型和 RAG 技术的高校知识库智能问答系统构建与评价[J].电脑知识与技术,2024,20(29):18-20+38.DOI:10.14004/j.cnki.ckt.2024.1449.
- [9] 蔡振华,王燕贞,杨净.基于 LangChain 和 ChatGLM 的高校财务问答系统研究与实现[J].现代计算机,2024,30(15):104-110.
- [10] Akkiraju R, Xu A, Bora D, et al. FACTS About Building Retrieval Augmented Generation-based Chatbots[J]. arXiv preprint arXiv:2407.07858, 2024.
- [11] 张玲玲,黄务兰.基于 ChatGPT API 和提示词工程的专利知识图谱构建[J/OL].情报杂志,1-8[2024-12-23].<http://kns.cnki.net/kcms/detail/61.1167.G3.20240828.1212.006.html>.
- [12] Liu P , Yuan W , Fu J ,et al.Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing[J]. 2021.DOI:10.48550/arXiv.2107.13586.

致 谢

行文至此，当论文落下最后一个句点，才知道学生时代即将远去。站在求学生涯的终点回望，那些在寝室挑灯的夜晚，在自习室查阅资料的周末，都成了青春里发光的印记。

感谢所有老师教诲，厦门理工学院的老师是我求学生涯遇到的最负责任最耐心最靠谱的老师。在这里特别感恩张旭导师，如同航海时的灯塔。在实习上提供的帮助、在论文上的严谨都是成就我不可或缺的关键因素。您们不仅教会我专业知识，更让我明白治学如做人，需脚踏实地。

同学好友身影始终清晰。记得无数个傍晚，在回寝室的路上，用不同理解和视角向往着对生活的期待。在项目合作上这份并肩作战的情谊，是大学时光最珍贵的馈赠。

父母二十载的守望是最温柔的铠甲。每次离家行李箱里塞满床单被铺，都是说不出口的牵挂；每次通话时你们总是嘘寒问暖、叮嘱，满是关心与问候。此刻，我将带着你们的期待走向更广阔的天地，那些未说出口的感激，都化作了前行的勇气。

最后要感谢这个充满可能的科技时代。当人工智能不再高高在上，我们有幸站在技术变革的潮头。特别感恩所有为科技发展奠基的前辈，是你们铺就的道路，让我们能够触摸星辰。

明理精工，与时俱进的校训早已融入血脉，祝愿母校越办越好。愿此去经年，永葆求知的热忱，不负韶华。